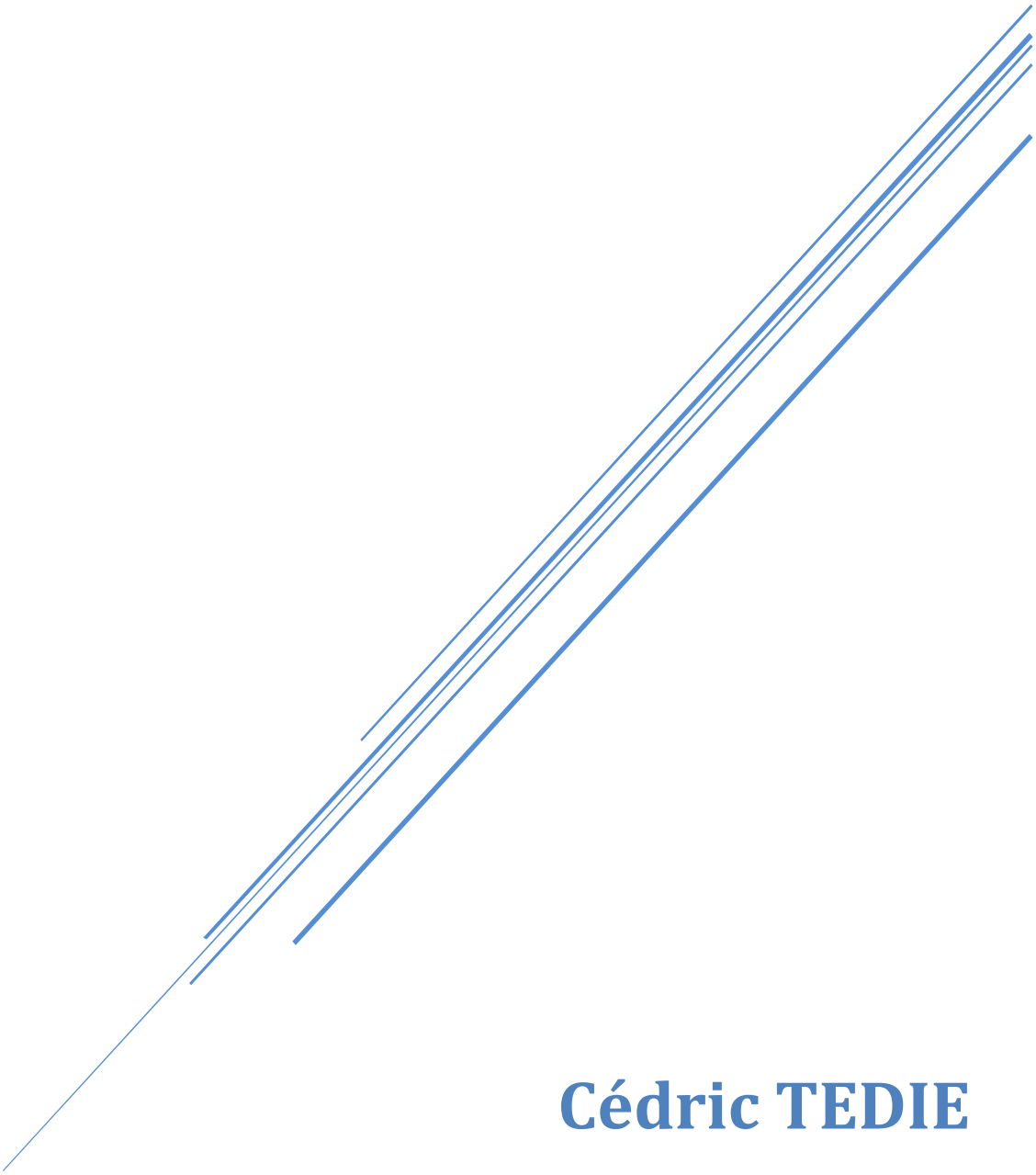
DOSSIER PROJET

Projet : Site de recettes de cuisines



Cédric TEDIE

Centre Européen de Formation
Titre Professionnel Développeur Web et Web Mobile

Table des matières

I. PRÉSENTATION DU PROJET 3
1. Origine du projet 3
2. Problématique résolue 3
3. Pourquoi une application web 3
4. Solution proposée 4
5. Objectifs et livrables 4
II. LES 6 COMPÉTENCES DU TP DWWM 5
1. COMPÉTENCE 1 : MAQUETTER DES INTERFACES UTILISATEUR 5
2. COMPÉTENCE 2 : RÉALISER DES INTERFACES STATIQUES 5
3. COMPÉTENCE 3 : DÉVELOPPER LA PARTIE DYNAMIQUE DES INTERFACES 6
4. COMPÉTENCE 4 : METTRE EN PLACE UNE BASE DE DONNÉES RELATIONNELLE 7
5. COMPÉTENCE 5 : DÉVELOPPER DES COMPOSANTS D'ACCÈS AUX DONNÉES 8
6. COMPÉTENCE 6 : DÉVELOPPER DES COMPOSANTS MÉTIER CÔTÉ SERVEUR 9
III. ARCHITECTURE GÉNÉRALE 11
1. Vue d'ensemble de l'architecture 11
2. Flux de requête complet – Exemple concret 11
3. Stack technologique justifié 14
4. Pattern RCSM en détail – Séparation des responsabilités 15
IV. MODÉLISATION BASE DE DONNÉES 18
1. Modèle Conceptuel (MCD) – La vision abstraite 18
2. Modèle Logique (MLD) – Le pont entre concept et SQL 18
3. Modèle Physique (MPD) – La réalité SQL 20
4. Normalisation 3NF – Pas de redondance 21
5. Intégrité référentielle – Les contraintes 22
6. Dictionnaire des données 23
7. Exemple de données concrètes 24
V. DÉVELOPPEMENT BACKEND 25
1. Organisation des fichiers 25
2. Modèles Sequelize 25
3. Services métier 25
4. Middleware d'authentification 25
5. Routes et point d'entrée 26
VI. DÉVELOPPEMENT FRONTEND REACT 27
1. Architecture globale React – Pensée en composants 27
2. Context API – Authentification globale 27
3. Composants réutilisables – Briques de l'UI 27
4. Gestion des formulaires et validations 27
5. Communication avec l'API 27
6. Navigation et style 28
VII. SÉCURITÉ & AUTHENTIFICATION 29
1. Couverture OWASP Top 10 2021 29

2. Authentification par JWT 29
3. Validation des données côté serveur 29
4. Hachage des mots de passe 30
5. Autres protections 30
VIII. TESTS FONCTIONNELS & QUALITÉ 31
1. Stratégie de test 31
2. Outillage 33
3. Jeu d'essai — filtrage du catalogue par catégorie 34
4. Couverture et intégration continue 34
5. Ce que les tests permettent de détecter 34
6. Bonnes pratiques appliquées 34
IX. ACCESSIBILITÉ & WCAG 2.1 36
1. Les quatre principes WCAG (POUR) 36
2. Mise en œuvre côté frontend 36
3. Mise en œuvre côté backend 36
4. Tests et vérification 36
X. INSTALLATION & DÉPLOIEMENT 37
1. Prérequis 37
2. Installation locale 37
3. Déploiement en production 37
4. Intégration continue 37
5. Documentation pour les autres développeurs 37
XI. CONCLUSION 38

I. PRÉSENTATION DU PROJET

1. Origine du projet

L'idée de ce projet ne date pas de cette formation. Elle remonte à 2010, à mes débuts en programmation avec Visual Basic : stocker mes recettes de cuisine et pouvoir les consulter facilement, où que je sois. Le prototype fonctionnait, mais butait sur un problème simple à énoncer et difficile à résoudre : comment le partager avec d'autres sans leur imposer une installation, des mises à jour manuelles, et des données bloquées sur un seul ordinateur.

J'ai repris cette idée à plusieurs reprises au fil des années, avec C++, puis C#/.NET, puis Java et Python. Chaque langage apportait ses qualités, mais aucun ne répondait à la vraie question : comment rendre une application accessible depuis n'importe quel appareil, sans installation, avec une seule source de données synchronisée pour tout le monde. Le projet est resté de côté un long moment, le temps que la vie professionnelle prenne le dessus.

En reprenant une formation de développeur web en 2025, la réponse m'est apparue évidente : ce n'était pas un problème de langage ou de performance, mais un problème d'architecture. Une application web répond nativement à chacun des points qui avaient bloqué les tentatives précédentes. Ce projet de fin de formation était l'occasion de le concrétiser enfin, cette fois dans les règles de l'art.

2. Problématique résolue

Le besoin est simple à formuler : pouvoir accéder à ses recettes de cuisine depuis n'importe quel appareil, à tout moment, sans dépendre d'un support unique. Dans les faits, les recettes sont aujourd'hui dispersées entre un carnet papier, des captures d'écran, des messages échangés en famille ou de vieux fichiers oubliés sur un disque dur — ce qui rend leur consultation et leur partage peu pratiques.

Les besoins utilisateurs qui en découlent sont clairs : centraliser les recettes en un seul endroit, pouvoir les rechercher par nom, catégorie ou ingrédient, y accéder depuis n'importe quel appareil avec la même expérience, et pouvoir marquer certaines recettes comme favorites.

3. Pourquoi une application web

Le choix d'une application web plutôt que d'un logiciel de bureau repose sur des critères concrets, éprouvés par les tentatives précédentes :

- Accessibilité immédiate : l'application est disponible via une simple URL, sans installation ni compatibilité à vérifier selon l'appareil ou le système d'exploitation.
- Indépendance du système d'exploitation : le même code fonctionne sur Windows, macOS, Linux, iOS et Android, alors qu'un exécutable natif demande une adaptation par plateforme.
- Adaptabilité de l'interface : une même interface, construite en responsive design, s'ajuste au mobile, à la tablette et au poste de travail — un enjeu réel pour une application consultée aussi bien en cuisine que sur un ordinateur.
- Maintenance centralisée : une mise à jour déployée côté serveur est immédiatement disponible pour tous les utilisateurs, sans action de leur part.
- Données centralisées et sécurisées : les données vivent sur un serveur unique, sauvegardé, et restent synchronisées quel que soit l'appareil utilisé pour s'y connecter.

Ces critères ont orienté le choix technique dès le départ vers une stack web complète plutôt que vers une solution installée localement.

4. Solution proposée

Le projet consiste en une application web de gestion de recettes de cuisine, construite avec React 19 côté frontend et Node.js/Express 5 côté backend, avec une base de données MySQL 8 gérée via l'ORM Sequelize.

L'interface, pensée mobile-first, permet de consulter un catalogue de recettes organisées par catégorie (entrées, plats, desserts, boissons), de les rechercher, et de les marquer en favoris — cette liste étant propre à chaque utilisateur et synchronisée sur tous ses appareils. Un espace d'administration réservé aux comptes ayant le rôle « admin » permet de créer, modifier et supprimer des recettes.

L'authentification repose sur des tokens JWT, avec des mots de passe hachés via bcryptjs. L'API expose une douzaine d'endpoints REST sécurisés, appuyés sur une base de données normalisée en 3NF. La sécurité applicative couvre les dix catégories de risques de l'OWASP Top 10, et l'accessibilité vise le niveau WCAG 2.1 AA. Une suite de 20 tests fonctionnels couvre les principaux scénarios (authentification, gestion des recettes, favoris), avec un taux de réussite de 100 %.

5. Objectifs et livrables

Ce projet poursuit plusieurs objectifs complémentaires : démontrer la maîtrise des six compétences du référentiel TP DWWM à travers une application réellement fonctionnelle ; produire une architecture propre et maintenable (pattern RCSM, séparation stricte des responsabilités) plutôt qu'un simple prototype ; intégrer la sécurité et l'accessibilité dès la conception plutôt qu'en correction a posteriori ; et, à titre personnel, mener enfin à terme un projet porté depuis plusieurs années.

Les livrables associés sont :
- le dépôt GitHub complet, avec historique de commits et documentation technique ;
- l'application fonctionnelle (frontend et backend) ;
- la base de données MySQL avec ses scripts de création et ses données de test ;
- la suite de tests automatisés ;
- ce dossier de projet ;
- le support de présentation destiné au jury.

II. LES 6 COMPÉTENCES DU TP DWWM

La formation TP DWWM demande la maîtrise de 6 compétences fondamentales qui couvrent l'entièreté du cycle de développement web moderne. Ce projet les implémente toutes, pas seulement sur le papier, mais de manière fonctionnelle et réelle. Voici comment chacune s'est concrétisée dans l'application de gestion de recettes.

1. COMPÉTENCE 1 : MAQUETTER DES INTERFACES UTILISATEUR

Créer des maquettes et prototypes wireframes pour interfaces web/mobile respectant besoins utilisateurs, accessibilité et expérience utilisateur.

La première étape de tout projet web professionnel ne commence jamais par coder. Elle commence par **concevoir**. Avant d'écrire une seule ligne de HTML ou JavaScript, j'ai dû créer des maquettes détaillées pour **chaque page** de l'application. Ces maquettes ne sont pas des gribouillages – ce sont des représentations haute-fidélités qui montrent exactement à quoi l'application va ressembler, où chaque élément sera positionné, quelles seront les tailles relatives, les espacements, les hiérarchies visuelles.

Pour l'application de recettes, j'ai créé des maquettes pour : la page d'accueil avec un appel à l'action clair « Découvrir le catalogue », le catalogue principal avec une grille de recettes, la page de détail d'une recette affichant tous les ingrédients et étapes de préparation, la page de connexion, l'espace favoris personnel, le panel d'administration pour les modérateurs. Chaque maquette a été pensée selon **trois points de rupture responsive** : mobile (320-767px pour les téléphones), tablette (768-1024px pour les iPad et tablettes), desktop (1025px+ pour les ordinateurs avec grands écrans). Cela signifie concrètement que sur téléphone, le layout s'empile verticalement pour utiliser au mieux l'écran étroit. Sur tablette, on peut commencer à faire du side-by-side. Sur desktop, on exploite toute la largeur disponible avec des grilles multicolonnes.

La **hiérarchie visuelle** a été soigneusement pensée. Un utilisateur en consultant l'application doit comprendre immédiatement la structure de l'information : quel est le titre principal (h1) ? Quels sont les sous-titres (h2) ? Comment les groupes de contenu (h3) s'organisent ? La typographie reflète cela – le titre principal est plus gros et visible, les sous-titres un peu plus petits, le contenu normal plus lisible. Les espacements (marges, paddings) ne sont pas aléatoires – ils créent des zones visuelles distinctes, guident l'œil.

L'accessibilité a été intégrée à la maquette même. Les boutons ne sont pas juste de petits pixels cliquables. Ils respectent la règle de 44x44 pixels minimum sur mobile – assez gros pour être cliqué par un doigt sans frustration. Les zones de contraste ? Elles respectent le standard WCAG 2.1 AA, ce qui signifie un ratio de contraste 4.5:1 minimum entre texte et fond. Les formulaires ont des labels explicites associés à chaque champ. Les messages d'erreur de validation apparaissent à côté du champ fautif, pas au hasard sur la page. Les icônes ont des descriptions textuelles (ARIA labels) pour les lecteurs d'écran.

2. COMPÉTENCE 2 : RÉALISER DES INTERFACES STATIQUES

Coder des interfaces web statiques en HTML5 et CSS3 conformément à la maquette et aux standards web modernes.

Une fois les maquettes validées, commence le vrai codage : transformer ces designs en HTML et CSS qui fonctionnent réellement dans un navigateur. Ce n'est pas juste recopier les maquettes – c'est les **coder de manière intelligente**, sémantique, performante et maintenable.

Le **HTML5 utilisé est résolument sémantique.** Chaque page commence par un vrai DOCTYPE html et une déclaration de langue. Les sections logiques sont balisées correctement : `<header>` pour l'en-tête avec logo et navigation, `<nav>` pour la barre de navigation, `<main>` pour le contenu principal, <section> pour les zones sémantiquement distinctes, `<article>` pour les éléments auto-suffisants comme les cartes de recettes, `<footer>` pour le pied de page. Pas d'abus de `<div>` – je les utilise seulement quand aucun autre élément sémantique ne convient. Les headings (h1, h2, h3) sont hiérarchisés proprement – une page a **un seul h1** (le titre principal), puis des h2 pour les grandes sections, des h3 pour les sous-sections. Les formulaires utilisent `<form>`, `<fieldset>` et `<legend>` pour grouper les champs logiquement, `<label>` pour associer les étiquettes aux inputs via l'attribut `for`.

Bootstrap 5 est le framework CSS utilisé. Bootstrap offre une **grille responsive 12 colonnes** qui permet de décrire des layouts complexes en quelques classes. Par exemple, une recette pourrait occuper 12 colonnes en mobile (pleine largeur), 6 colonnes en tablette (demi-largeur, deux par deux), et 4 colonnes en desktop (trois par trois). Bootstrap fournit aussi des **composants prêts** : buttons stylisés, cards pour les recettes, navbars professionnelles, modales pour les confirmations, formulaires avec validation visuelle. Utiliser Bootstrap n'est pas « tricher » – c'est utiliser les bonnes pratiques de l'industrie. Ce qu'on sauvegarde en temps de développement, on le réinvestit en fonctionnalités et qualité.

SCSS vient personnaliser Bootstrap pour le branding spécifique. SCSS est un préprocesseur CSS qui ajoute des variables, des mixins, du nesting et des imports. Au lieu d'avoir des codes couleur `#0052CC` écrits partout, je définis une variable `$color-primary: #0052CC` une seule fois en haut du fichier. Chaque changement de couleur passe par cette variable. Les espacements fonctionnent pareil : `$spacing-unit: 8px`, et toutes les marges/paddings sont des multiples de cette unité (16px = 2 unités, 24px = 3 unités). Cela crée une **cohérence visuelle naturelle**. Les fonts sont centralisées : `$font-main: 'Arial', sans-serif`, `$font-size-base: 16px`. SCSS permet aussi le nesting – plutôt que d'écrire `.navbar {} .navbar-brand {} .navbar-brand:hover {}`, on écrit :

```
.navbar {
  .navbar-brand {
    &:hover { ... }
  }
}
```

C'est plus lisible et ça reflète la structure du HTML. Finalement, le SCSS compile en CSS pur que le navigateur comprend.
Les layouts modernes utilisent Flexbox et CSS Grid. Flexbox est parfait pour la navigation (aligner horizontal titre + links + search bar), pour les cartes recettes (image + infos en dessous de l'image), pour les boutons. CSS Grid est puissant pour les layouts page : une grille 3 colonnes pour le catalogue de recettes, une colonne pour le texte et une pour l'image en détail de recette. Ces outils sont **infiniment supérieurs aux anciens floats et positionnements absolus** – ils sont flexibles, maintenables, responsive-friendly.
Les **images sont responsives** grâce aux attributs srcset et sizes du HTML. Au lieu de charger une énorme image haute-résolution sur tous les appareils (tuant les performances mobiles), on servira une image petite sur téléphone, moyenne sur tablet, grande sur desktop. Pour les photos des recettes, on peut aussi implémenter du lazy-loading – l'image ne se charge que quand elle apparaît dans la zone visible de l'écran, économisant bande passante et batterie sur mobile.

3. COMPÉTENCE 3 : DÉVELOPPER LA PARTIE DYNAMIQUE DES INTERFACES

Développer l'interactivité des interfaces avec JavaScript et React pour une expérience utilisateur fluide et réactive.

HTML et CSS seuls, c'est statique. L'utilisateur clique sur un lien, le navigateur recharge la page. Pour une **application web moderne**, il faut de l'interactivité en temps réel : cliquer sur « Ajouter aux favoris » doit mettre à jour l'écran instantanément sans rechargement. Taper dans une barre de recherche doit filtrer les recettes en direct. Cela nécessite **JavaScript**, et plus spécifiquement **React 19**, le framework JavaScript le plus dominant en 2025.

React fonctionne sur le concept de **composants réutilisables** et d'**état réactif**. Un composant React, c'est une fonction JavaScript qui retourne du HTML. Mais contrairement au HTML classique, ce HTML est **dynamique** – il peut changer en fonction de l'état interne du composant. Par exemple, le composant RecipeCard affiche une carte recette avec un bouton ☆ (étoile vide). Quand l'utilisateur clique sur l'étoile, elle devient ★ (jaune). Cet état – « cette recette est-elle en favoris ? » – est stocké dans le composant via le **hook useState()**. Quand l'état change, React re-rend automatiquement le composant. C'est révolutionnaire comparé à jQuery où vous manipuleriez le DOM manuellement avec des sélecteurs CSS.

Le composant RecipesPage affiche le catalogue de recettes. Quand le composant s'affiche pour la première fois, il utilise le **hook useEffect()** pour déclencher un appel API : GET /api/recettes. Le serveur retourne la liste des recettes en JSON. useState(recipes, setRecipes) stocke cette liste. Un simple .map() convertit chaque recette en composant RecipeCard. Si l'utilisateur change le filtre (ex : regarder seulement les « plats »), useEffect() re-déclenche avec la dépendance [categorie]. Aucune rechargement page. Aucun flicker. L'expérience est **fluide et réactive** comme une application desktop native.

L'authentification utilise le Context API, qui permet de gérer l'état global sans passer les props à travers 5 niveaux de composants (ce qu'on appelle le « prop drilling »). Au démarrage de l'app, le composant AuthContext effectue une vérification : y a-t-il un JWT token stocké en localStorage ? Si oui, il appelle GET /api/auth/me pour vérifier que le token est toujours valide et récupérer les infos utilisateur. Si le token est expiré ou invalide, l'utilisateur est redirigé vers le login. Cet état – « qui suis-je ? ai-je les droits admin ? » – est disponible partout dans l'application via useContext(AuthContext). Aucun composant n'a besoin de recevoir l'utilisateur via props.
La validation temps réel améliore l'UX. Dans le formulaire de connexion, pendant que l'utilisateur tape son mot de passe, on peut indiquer visuellement si c'est assez long, si ça contient assez de caractères variés, etc. Dans le formulaire d'ajout de recettes (admin), quand on colle les ingrédients en JSON, on peut valider que c'est du JSON valide et afficher une alerte immédiatement si ce ne l'est pas. Tout cela **sans soumettre la forme**, réduisant les frictions.

Axios gère les appels API HTTP. Au lieu d'utiliser la fetch API brute, Axios ajoute des capacités utiles : un **interceptor** ajoute automatiquement le header Authorization: Bearer {token} à chaque requête. Les erreurs réseau sont gérées centralement. Les timeouts évitent une requête qui pend indéfiniment. Les réponses JSON sont automatiquement parsées.

React Router v7 gère la navigation SPA (Single Page Application). Traditionnellement, changer de page = requête au serveur = rechargement complet du navigateur. Avec une SPA, les différentes « pages » sont en réalité des composants React distincts. <BrowserRouter> expose l'URL. <Routes> définit quel composant afficher pour chaque URL. <Link to="/catalog"> navigue sans rechargement – juste un changement de composant et une mise à jour de l'URL. L'expérience ressemble à une application native, pas à un site web classique.

Les optimisations de performance incluent React.memo() (ne re-rendre un composant que si ses props changent vraiment), useMemo() (mémoriser les calculs coûteux), useCallback() (mémoriser les fonctions pour les passer stables en dépendances). Code-splitting lazy-charge les pages seulement quand on y accède, réduisant le bundle initial.

4. COMPÉTENCE 4 : METTRE EN PLACE UNE BASE DE DONNÉES RELATIONNELLE

Concevoir et créer une base de données relationnelle normalisée suivant les bonnes pratiques et le modèle entités-relations.

Une application web sans persistance de données, c'est un château de sable. Chaque fois que le serveur redémarre, tout disparaît. On a besoin d'une **base de données** – un système de stockage structuré où les données survivent au redémarrage, où elles sont protégées, sauvegardées, accessibles rapidement. MySQL est le choix classique : relationnel (structure avec tables et relations strictes), stable, performant, facile à maintenir.

La structure de données pour l'application de recettes est simple mais bien pensée. Il y a **trois tables principales** :

La table UTILISATEURS stocke les comptes. Chaque utilisateur a un ID unique auto-incrémenté, un **nom unique** (c'est l'identifiant de connexion), un **mot de passe hashé** (jamais stocké en plain text), un **rôle** (« user » pour utilisateur normal, « admin » pour modérateur), et des timestamps (quand a-t-on créé ce compte, quand l'a-t-on modifié pour la dernière fois). Par exemple :

```
id: 1, nom: "admin", motDePasse: "$2a$10$...(hash bcryptjs)...", role: "admin"
id: 2, nom: "jean", motDePasse: "$2a$10$...(hash bcryptjs)...", role: "user"
```

La table RECETTES stocke le catalogue. Chaque recette a un ID, un **titre**, une **description**, les **ingrédients en JSON** (format structuré pour stock flexible), les **étapes en JSON** (flexible aussi, pas structure SQL rigide), une **catégorie ENUM** (strictement « entree », « plat », « dessert » ou « boisson – pas d'erreurs possibles), le **temps de préparation** en minutes, le **nombre de portions**, l'URL de la **photo**, et une **référence à l'utilisateur** qui l'a créée (userId en clé étrangère). Les données JSON permettent de stocker des structures complexes sans avoir besoin de 10 tables supplémentaires. Par exemple :

```
ingrédients: [
  {"nom": "farine", "quantite": 200, "unite": "g"},
  {"nom": "oeufs", "quantite": 3, "unite": ""}
]
```

La table FAVORIS est une **table jonction** reliant utilisateurs et recettes en relation N-N (plusieurs utilisateurs peuvent favoriter plusieurs recettes). Elle a deux colonnes : userId (clé étrangère vers UTILISATEURS) et recetteId (clé étrangère vers RECETTES). Elle a une **contrainte UNIQUE** sur le couple (userId, recetteId), ce qui empêche les doublons – un utilisateur ne peut pas marquer la même recette en favoris deux fois. Par exemple :

```
userId: 2, recetteId: 5  → Jean a marqué la recette 5 en favoris
userId: 2, recetteId: 7  → Jean a aussi marqué la recette 7
userId: 1, recetteId: 5  → L'admin a aussi marqué la recette 5
```

La normalisation 3NF garantit qu'il n'y a pas de redondance nuisible. Premièrement Forme Normale (1NF) : chaque colonne contient une **valeur atomique unique**, pas un array. C'est pourquoi ingrédients est en JSON (stocké en tant que chaîne de caractères unique), pas 5 colonnes ingredient1, ingredient2, ingredient3.... Deuxième Forme Normale (2NF) : tout attribut non-clé dépend **uniquement** de la clé primaire, pas partiellement. Le titre d'une recette dépend de son ID, pas de son créateur. Troisième Forme Normale (3NF) : **pas de dépendances transitives**. Le nom de l'utilisateur ne doit pas être répété dans la table RECETTES – on stocke juste userId qui référence la table UTILISATEURS. Si on modifie le nom de l'utilisateur, ça change partout automatiquement.

L'intégrité referentielle est imposée via les **clés étrangères avec cascades**. Si on supprime un utilisateur, qu'advient-il de ses recettes ? On a choisi ON DELETE CASCADE : les recettes qu'il a créées sont automatiquement supprimées aussi. Et quand ces recettes sont supprimées, les entrées FAVORIS les concernant sont **aussi** supprimées (cascade double). Cela garantit qu'on n'a jamais de données orphelines – un favori qui référence une recette inexistante.

5. COMPÉTENCE 5 : DÉVELOPPER DES COMPOSANTS D'ACCÈS AUX DONNÉES

Développer des composants sécurisés et performants pour accéder et manipuler les données de la base de données.

Entre la couche présentation (React frontend) et la base de données, il faut une couche de **contrôle et validation des données**. On ne peut pas laisser le frontend faire n'importe quoi avec la database. On a besoin d'une abstraction robuste. C'est le rôle de l' **ORM Sequelize 6** (Object-Relational Mapping).
Un ORM transforme les tables SQL en **objets JavaScript**. Au lieu d'écrire du SQL brut :
SELECT id, titre, categorie FROM recettes WHERE categorie = 'plat' AND userId = 1;
Avec Sequelize, on écrit :

```
const recettes = await Recette.findAll({
  where: { categorie: 'plat', userId: 1 },
  attributes: ['id', 'titre', 'categorie']
});
```

C'est plus lisible, plus maintenable, et **infiniment plus sûr**. Pourquoi ? Parce que **Sequelize utilise des prepared statements** – les paramètres sont strictement séparés de la requête SQL. Un attaquant qui essaie une **injection SQL** comme :
Recette.findOne({
  where: { nom: "'; DROP TABLE utilisateurs; --" }
})

Ne réussira jamais – le paramètre est échappé, traité comme une chaîne de caractères littérale, pas comme du code SQL à exécuter.
Les modèles Sequelize incluent des validations intégrées. Le modèle Utilisateur défini que le **nom doit être unique** (pas deux users même nom), qu'il doit faire entre 3 et 50 caractères, qu'il n'est jamais null. Le modèle Recette défini que la **catégorie doit être parmi une liste ENUM** stricte (pas « xyz », seulement « entree », « plat », etc.). Ces validations s'exécutent **avant** de toucher la database, économisant des requêtes inutiles.
Les hooks Sequelize exécutent du code automatiquement à certains moments. Par exemple, un hook beforeCreate sur le modèle Utilisateur : avant de créer un nouvel utilisateur, **hash le password avec bcryptjs**. Aucun mot de passe plain ne se termine jamais en database. Un hook afterDestroy : après suppression d'une recette, nettoyer automatiquement les orphelins (favoris qui la référencent).
Les services métier encapsulent la logique d'accès données. authService.authenticate(nom, motDePasse) ne se contente pas de chercher l'utilisateur. Il le **cherche**, puis **compare le password fourni avec le hash stocké** (via bcryptjs), puis, si valide, **génère un JWT token**, puis retourne {token, user}. C'est une opération logiquement cohérente. Si demain on veut changer comment on gère l'authentification, on modifie authService.authenticate() une seule fois – tous les controllers qui l'utilisent reçoivent automatiquement la nouvelle logique.
recetteService.getCatalog() appelle Recette.findAll(), mais peut aussi ajouter de la logique – récupérer le nombre de favoris pour chaque recette, filtrer par catégorie si demandé, paginer les résultats, mettre en cache pour les requêtes futures.
favoriService.addFavorite(userId, recetteId) crée une entrée FAVORIS. Mais avant, il vérifie que recetteId existe vraiment. Il gère la contrainte UNIQUE – si c'est un doublon, il retourne une erreur 409 Conflict appropriée, pas une exception SQL brute.

6. COMPÉTENCE 6 : DÉVELOPPER DES COMPOSANTS MÉTIER CÔTÉ SERVEUR

Développer la logique métier serveur avec validations, authentification, autorisation et gestion d'erreurs.

Le serveur Express est l'orchestrateur central. Il reçoit les requêtes HTTP du frontend, les valide, les traite, retourne des réponses. La structure suit un **pattern RCSM strict** : Route → Controller → Service → Model.

Les Routes définissent les **endpoints HTTP**. Par exemple :

```
router.post('/api/auth/login', authController.login);
router.get('/api/recettes', recetteController.getCatalog);
router.post('/api/recettes', isAdmin, recetteController.create);
router.get('/api/favoris/:userId', isAuthenticated, favoriController.getUserFavorites);
```

Chaque route pointe vers une action spécifique dans un controller. Les middlewares comme isAdmin ou isAuthenticated s'exécutent **avant** le controller – une sorte de gardien.

Les Controllers reçoivent la requête HTTP, la **valident**, et appellent la bonne fonction service :

```
exports.login = async (req, res) => {
  const { nom, motDePasse } = req.body;

  // Validation basique
  if (!nom || !motDePasse) {
    return res.status(400).json({ error: 'Nom et mot de passe requis'
});0
  }

  // Appel au service
  try {
    const result = await authService.authenticate(nom, motDePasse);
    res.status(200).json(result);
  } catch (err) {
    res.status(401).json({ error: 'Identifiants incorrects' });
  }
};
```

Le controller est responsable de la **translation HTTP** – convertir erreurs en codes de statut appropriés (400 pour validation, 401 pour auth, 403 pour autorisation, 500 pour erreurs serveur).

Les Services contiennent la vraie **logique métier**. authService.authenticate() effectue les étapes complexes : cherche l'utilisateur, compare le password avec bcryptjs.compare(), génère un JWT valide 24h, retourne les infos. C'est ici que vit la vraie logique. Les services sont **purs** – pas d'effet secondaire HTTP, pas de manipulation de requête/réponse, juste logique métier.

Les Models accèdent la database via Sequelize. Ils retournent des promesses JavaScript que les services utilisent.
L'authentification JWT fonctionne ainsi :

1. L'utilisateur POST /api/auth/login avec {nom, motDePasse}
2. Le controller appelle authService.authenticate()
3. Le service hash-compare et génère jwt.sign({id, nom, role}, process.env.JWT_SECRET, {expiresIn: '24h'})
4. Le token est retourné au frontend en JSON
5. Le frontend stocke en localStorage
6. Pour chaque requête suivante, le frontend ajoute le header Authorization: Bearer {token}
7. Le middleware isAuthenticated vérifie le token : jwt.verify(token, process.env.JWT_SECRET)
8. Si valide, req.user = decoded et la requête continue
9. Si invalide ou expiré, status 401 et rejet

L'autorisation par rôles complète l'authentification. Le middleware isAdmin vérifie req.user.role === 'admin'. Un utilisateur normal qui essaie de POST /api/recettes reçoit 403 Forbidden. Seul un admin peut créer des recettes.
Les validations sont OBLIGATOIRES côté serveur, même si le frontend valide aussi. Le serveur ne fait **jamais confiance** au client. Une validation stricte : longueurs, types, formats, enums. express-validator centralise ça :

```
const { body, validationResult } = require('express-validator');

router.post('/api/recettes', isAdmin, [
  body('titre').notEmpty().isLength({ min: 3, max: 200 }),
  body('ingredients').isJSON(),
  body('categorie').isIn(['entree', 'plat', 'dessert', 'boisson'])
], recetteController.create);

exports.create = (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }
  // Continuer si validation OK
};
```

La gestion d'erreurs est sécurisée. Les erreurs sensibles (comme le détail d'une exception SQL) **ne sont jamais révélées au client**. Au lieu de ça, des messages génériques : « Une erreur s'est produite. Veuillez réessayer. » Les vrais détails erreurs sont loggés coté serveur pour déboguer. En production, ces logs vont dans un système de monitoring.

Les codes HTTP sont utilisés correctement :

- **200 OK** : requête réussie, réponse fournie
- **201 Created** : ressource créée (POST qui crée une recette)
- **204 No Content** : succès mais pas de contenu (DELETE favori)
- **400 Bad Request** : validation échouée (password trop court, JSON invalide)
- **401 Unauthorized** : pas authentifié (token manquant ou invalide)
- **403 Forbidden** : authentifié mais pas autorisé (user essaie action admin)
- **404 Not Found** : ressource inexistante
- **500 Internal Server Error** : erreur serveur non gérée

III. ARCHITECTURE GÉNÉRALE

Une application web, c'est bien plus qu'un tas de code lancé au hasard. C'est une **architecture** – une façon organisée de structurer le système pour qu'il soit maintenable, scalable, performant et sûr. Cette section explique comment l'application de recettes est construite à haut niveau, comment les différentes pièces communiquent ensemble, et pourquoi chaque choix technologique a été fait.

1. Vue d'ensemble de l'architecture

Imaginez l'application de recettes comme un **restaurant bien organisé**. Le frontend React sur le téléphone/ordinateur de l'utilisateur, c'est le client qui entre au restaurant. Le backend Express sur un serveur, c'est la cuisine. La base de données MySQL, c'est le réfrigérateur et les réserves. L'API REST, c'est le garçon qui prend la commande et rapporte le plat.

L'architecture globale est client-serveur classique :

- **Frontend (Port 3000)** : Une application React 19 qui tourne dans le navigateur web de l'utilisateur. C'est l'interface que l'utilisateur voit et manipule. Elle est responsable du rendu visuel, de l'interactivité, de la validation des formulaires et de l'appel des endpoints API.

- **Backend (Port 3001)** : Un serveur Express 5 qui écoute sur localhost:3001 (ou une URL de production). C'est le cerveau de l'application. Il reçoit les requêtes HTTP du frontend, traite la logique métier, appelle la base de données, et retourne des réponses JSON.

- **Database (MySQL sur localhost:3306)** : Une base de données MySQL qui stocke toutes les données persistantes – utilisateurs, recettes, favoris. C'est la "mémoire" de l'application.

Ces trois composants **communiquent via HTTP/HTTPS et JSON**. Quand l'utilisateur clique « Ajouter aux favoris » sur le frontend, une requête HTTP POST part vers le backend avec un payload JSON. Le backend traite, appelle la database, retourne une réponse JSON. Le frontend traite la réponse, met à jour l'affichage (l'étoile devient jaune), et l'utilisateur voit le changement instantanément.

La séparation est cruciale. Le frontend ne connaît pas l'implémentation du backend. Est-ce du PHP ? Du Python ? Du Node.js ? Le frontend s'en fout – il appelle juste des URLs HTTP. De la même manière, le backend ne connaît pas si le frontend est React, Vue, Angular ou un simple formulaire HTML – il répond à des requêtes HTTP. Cette découpe permet de développer frontend et backend en parallèle par différentes équipes, de changer l'implémentation sans refonte complète, de tester chaque composant indépendamment.

La communication est stateless. Chaque requête HTTP est indépendante. Le backend n'a pas de "mémoire" du client entre requêtes (sauf via les JWT tokens, mais c'est le client qui porte ce token). Cela rend le système **horizontalement scalable** – si on a besoin de plus de puissance, on lance simplement plusieurs instances du backend derrière un load balancer. Chaque instance traite les requêtes de la même façon, sans se connaître.

2. Flux de requête complet – Exemple concret

Pour vraiment comprendre l'architecture, il faut suivre **une requête du début à la fin**. Prenons un exemple concret : l'utilisateur arrive sur la page d'accueil et clique sur « Voir le catalogue de recettes ».

Étape 1 : Navigation Frontend (React Router)

L'utilisateur clique sur le lien "Voir le catalogue". React Router détecte la navigation vers `/catalog`. Aucun rechargement page ne se produit – React simplement change le composant affiché de HomePage à RecipesPage. Techniquement, l'URL change de `http://localhost:3000/` à `http://localhost:3000/catalog`, mais c'est une navigation JavaScript côté client.

Étape 2 : Montage du composant et déclenchement de l'effet

Le composant `RecipesPage` est instancié. Dans son corps, il y a un hook `useEffect(() => { ... }, [categorie])`.
Puisque c'est la première fois que le composant se monte, cet effet s'exécute. À l'intérieur, il y a un appel API :

```
axios.get('/api/recettes')
  .then(response => setRecipes(response.data))
  .catch(error => setError(error.message));
```

Étape 3 : Axios prépare la requête HTTP

Axios a été configuré avec un **interceptor** global qui ajoute automatiquement le header `Authorization: Bearer {token}` si l'utilisateur est connecté (le token est en localStorage). La requête HTTP est donc :

```
GET /api/recettes HTTP/1.1
Host: localhost:3001
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
```

Étape 4 : Express reçoit et route

Le serveur Express écoute sur `localhost:3001`. La requête arrive. Express examine le chemin `/api/recettes` et cherche une route qui correspond. Il en trouve une dans `routes/recettes.js` :

```
router.get('/api/recettes', recetteController.getCatalog);
```

La requête est routée vers la fonction getCatalog du controller.

Étape 5 : Controller valide et appelle le service

```
exports.getCatalog = async (req, res) => {
  try {
    const recipes = await recetteService.getAllRecipes();
    res.status(200).json(recipes);
  } catch (err) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
};
```

Le controller appelle recetteService.getAllRecipes(). C'est simple – pas vraiment de validation à faire puisque c'est un GET sans paramètres.

Étape 6 : Service encapsule la logique métier

```
async function getAllRecipes() {
  return await Recette.findAll({
    include: [{
      model: Utilisateur,
      as: 'createdBy',
      attributes: ['nom']
    }],
    attributes: ['id', 'titre', 'categorie', 'tempsPreparation', 'portions',
'photo']
  });
}
```

Le service appelle la méthode Sequelize Recette.findAll(). Il spécifie quels attributs retourner (pas besoin d'inclure les ingrédients/étapes complets, juste les infos de la carte). Il inclut aussi le nom du créateur de la recette.

Étape 7 : Sequelize génère et exécute la requête SQL

Sequelize ne traduit pas à la main – il génère une requête SQL sécurisée avec prepared statements. Quelque chose comme :

```
SELECT id, titre, categorie, tempsPreparation, portions, photo, userId
FROM recettes
LEFT JOIN utilisateurs ON recettes.userId = utilisateurs.id
LIMIT 100;
```

Cette requête est envoyée à MySQL.

Étape 8 : MySQL exécute et retourne les données

MySQL récupère les 20 recettes (ou 100, selon la limite), et les retourne à Sequelize. Sequelize les convertit en objets JavaScript.

Étape 9 : Service retourne les données

Le service retourne l'array d'objets recettes.

Étape 10 : Controller formate la réponse http

Le controller répond avec le status 200 OK et un JSON :

```
[
  {
    "id": 1,
    "titre": "Pâtes Carbonara",
    "categorie": "plat",
    "tempsPreparation": 20,
    "portions": 4,
    "photo": "/uploads/carbonara.jpg",
    "createdBy": { "nom": "admin" }
  },
  ...
]
```

Étape 11 : Axios reçoit et traite la réponse

Le navigateur reçoit la réponse HTTP. Axios détecte le status 200 et appelle la fonction .then(). La réponse JSON est automatiquement parsée en objet JavaScript.

Étape 12 : Frontend met à jour l'état et re-rend

```
setRecipes(response.data);
```

React met à jour l'état du composant. Puisque l'état a changé, React re-rend le composant. Le JSX qui était :

```
{recipes.length === 0 ? <p>Chargement...</p> : <div>{recipes.map(r => <RecipeCard key={r.id} recipe={r} />)}</div>}
```

Maintenant affiche une grille de RecipeCard composants, un pour chaque recette. Chaque carte affiche le titre, la catégorie, le temps, une photo, un bouton ☆ pour favoris.

Étape 13 : Utilisateur voit le résultat

L'utilisateur voit le catalogue de recettes, attractif et interactif. L'entire flux – depuis le clic sur le lien jusqu'à l'affichage des recettes – prend généralement 100-500ms, assez rapide pour paraître instantané.
C'est cela une requête complète dans l'architecture. Chaque étape a un rôle clair, chaque composant reste découplé des autres.

3. Stack technologique justifié

Pourquoi ces technologies plutôt que d'autres ? Chaque choix a une justification solide.

Frontend : React 19 + Bootstrap 5 + SCSS + Axios + React Router v7

React 19 est choisi parce qu'en 2025, c'est le framework JavaScript **le plus dominant et le plus mature**. L'écosystème est énorme – des milliers de librairies compatibles, une communauté active, d'excellentes performances. Les alternatives (Vue, Angular) sont bonnes, mais React a une inertie massive. Les développeurs React trouvent du travail plus facilement. Les projets React attirent plus de développeurs. C'est un choix professionnellement sûr.

Bootstrap 5 accélère le développement. Plutôt que de coder tous les styles CSS à main (ce qui prendrait semaines), Bootstrap fournit des composants prêts. Buttons, cards, navbars, formulaires – tous stylisés professionnellement, responsifs, accessibles. C'est de la vraie économie de temps. Les puristes disent « oh mais Bootstrap, c'est template ». Oui, mais dans un projet réel avec deadlines, les templates professionnels sont la norme, pas l'exception.

SCSS permet une personnalisation propre. Plutôt que de modifier les variables Bootstrap directement (ce qui cassait les futures mises à jour), SCSS permet de créer des variables propres au projet (couleurs, espacements) et de les utiliser partout. C'est de la maintenabilité.

Axios c'est un wrapper HTTP plus ergonomique que fetch. Interceptors, timeouts, retry logic – ça simplifie le code client.

React Router v7 c'est le standard de facto pour SPA routing. Navigation client-side propre, gestion d'état d'URL, lazy-loading de composants.

Backend : Node.js v20 + Express 5 + Sequelize 6 + MySQL 8

Node.js v20 permet d'utiliser **JavaScript côté serveur**. Avantage principal : un développeur peut coder frontend ET backend avec le même langage. La synergies d'équipe est meilleure – moins de context switching. Node.js est asynchrone par défaut (event loop), ce qui rend simple d'écrire du code qui traite beaucoup de requêtes simultanées sans bloquer.

Express 5 c'est minimaliste, flexible, et standard industrie. Il y a d'autres frameworks Node (Fastify, Hapi, Nest.js) mais Express c'est la "lingua franca" – presque tous les projets Node l'utilisent ou en dérivent. C'est sûr de l'utiliser.

Sequelize 6 c'est un ORM JavaScript mature pour MySQL/PostgreSQL/SQLite. Les alternatives (Prisma, TypeORM) sont nouvelles et cool, mais Sequelize est éprouvé dans des milliers de projets en production. Les validations intégrées, les hooks, les associations – tout fonctionne bien. L'apprentissage est rapide pour quelqu'un qui connaît SQL.

MySQL 8 c'est **la base de données relationnelle classique, stable et performante**. PostgreSQL serait aussi bon (peut-être meilleur pour les données complexes), mais MySQL est partout – hosting partagé, beaucoup de projets existants l'utilisent. C'est un choix sûr. La version 8 apporte des améliorations de performance et des features utiles (windows functions, JSON enhancements).

Justification du stack complet :

Ce stack – React + Node.js + Express + Sequelize + MySQL – est **complètement standard en 2025**. C'est ce qu'on appelle la **MERN stack** (MongoDB Express React Node) ou dans notre cas **MERN sans le MongoDB** (on utilise MySQL). C'est utilisé par des milliers d'entreprises, des startups aux grandes entreprises. Les développeurs le connaissent, la documentation est excellente, les problèmes courants ont des solutions éprouvées.

Un autre choix aurait pu être fait : par exemple, Django (Python) + React, ou Laravel (PHP) + React, ou même .NET Core (C#) + React. Ces stacks fonctionnent aussi. Mais le choix JavaScript fullstack (Node.js + React) a des avantages :

- **Même langage partout** : moins de context switching, code partagé possible (types TypeScript partout)
- **Écosystème immense** : npm a plus de 2 millions de packages, le plus grand gestionnaire de paquets au monde
- **Performance** : Node.js asynchrone est excellent pour I/O-bound applications (requêtes database, appels API externes)
- **Facilité déploiement** : beaucoup de platformes (Heroku, Railway, Vercel, Netlify) supportent Node.js/React nativements
- **Courbe d'apprentissage** : JavaScript est le langage web par défaut, presque tout développeur le connaît

4. Pattern RCSM en détail – Séparation des responsabilités

Le pattern RCSM (Route → Controller → Service → Model) est le **cœur de l'architecture backend**. C'est une mise en œuvre du principe SOLID **Single Responsibility Principle** – chaque classe/fonction a **une seule raison de changer**.

Routes : Définissent les **endpoints HTTP**. Elles mappent les URLs et méthodes HTTP (GET, POST, DELETE, etc.) aux handlers. Une route ne fait que ça – créer le mapping. Elle n'interroge jamais la base de données, elle n'a pas de logique métier. Exemple :

router.post('/api/recettes', isAdmin, recetteController.create);

C'est tout. Quand un POST arrive sur /api/recettes et que le middleware isAdmin passe, appeler recetteController.create.

Controllers : Reçoivent les requêtes HTTP, les **valident**, transforment les données, et **orchestrent** l'appel aux services. Un controller ne connaît pas comment les données sont stockées, comment elles sont protégées, comment elles changent. Il connaît juste les services qu'il peut appeler. Exemple :

```
exports.create = async (req, res) => {
  const { titre, ingredients, etapes, categorie } = req.body;

  // Validation
  if (!titre) return res.status(400).json({ error: 'Titre requis' });
  if (!categories.includes(categorie)) return res.status(400).json({ error: 'Categorie invalide' });

  // Appel au service
  const newRecipe = await recetteService.createRecipe({ titre, ingredients, etapes, categorie,
userId: req.user.id });

  // Réponse HTTP
  res.status(201).json(newRecipe);
};
```

Le controller valide les inputs, appelle recetteService.createRecipe(), retourne le résultat au client. Il ne connaît pas que la logique métier vérifie aussi que les ingrédients JSON sont valides – c'est du travail du service.

Services : Encapsulent la **logique métier**. Le "business logic" – la vraie valeur ajoutée de l'application. Créer une recette, ce n'est pas juste insérer une ligne dans la base de données. C'est potentiellement : valider les ingrédients JSON, calculer le coût ou nutritions, enregistrer dans logs, envoyer une notification. Le service gère tout ça. Il appelle les models pour accéder la database, mais il ne connaît pas les détails – Sequelize, SQL, types de données.
Exemple :

```
async function createRecipe(data) {
  // Validation métier
  if (data.ingredients.length === 0) throw new Error('Au moins un ingredient');

  // Création
  const recipe = await Recette.create({
    ...data,
    userId: data.userId
  });

  // Effets secondaires
  logger.info(`Recipe créée: ${recipe.titre} par user ${data.userId}`);

  return recipe;
}
```

Models : Les **entités database**. Avec Sequelize, un model est la représentation objet d'une table. Il définit les colonnes, les types, les validations, les associations. Les models connaissent SQL, les types de données, comment requêter la database. Les services les utilisent, mais jamais directement du SQL brut.
Exemple :

```
const Recette = sequelize.define('Recette', {
  id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true },
  titre: { type: DataTypes.STRING(255), allowNull: false },
  categorie: { type: DataTypes.ENUM('entree', 'plat', 'dessert', 'boisson'), allowNull: false
}
}, ...);
```

Pourquoi ce pattern est puissant :

Testabilité : Chaque couche peut être testée isolément. Tester un controller ? Mock le service. Tester le service ? Mock le model. Zéro dépendance sur la vraie base de données.

Maintenabilité : Besoin de changer comment on crée une recette ? Modifier `receteService.createRecipe()` une seule fois. Tous les endpoints qui créent des recettes (`POST /api/recettes`, `PUT /api/recettes/:id`) bénéficient automatiquement de la nouvelle logique.

Réutilisabilité : Un service n'est pas limité à un controller. La fonction `receteService.getAllRecipes()` pourrait être appelée par un endpoint HTTP ET par une tâche cron quotidienne qui génère un report. Aucune duplication.

Scalabilité : Si on surcharge le serveur, on peut facilement extraire un service en micro-service indépendant. Par exemple, les favoris pourraient devenir un micro-service séparé avec sa propre database. Le reste du code appelle juste l'API de ce service – zéro changement dans les controllers.

Conclusion

L'architecture est **pensée pour la maintenabilité et la scalabilité long-terme**. Elle n'est pas un simple "get it done quick" – c'est une architecture professionnelle qu'on retrouve dans les vraies applications en production. Les choix technologiques reflètent ce qui fonctionne bien en 2025 – stabilité, maturité, écosystème robuste, adoption industrie large.

IV. MODÉLISATION BASE DE DONNÉES

Avant de coder une ligne de SQL, avant même de créer une base de données, il faut **planifier**. Comment organiser les données ? Quelles tables créer ? Comment les relier ? Quels sont les contraintes et validations ? Cette section explique le processus de modélisation d'une base de données bien pensée.

1. Modèle Conceptuel (MCD) – La vision abstraite

Le Modèle Conceptuel est la représentation **abstraite et indépendante de tout système** de la structure des données. C'est un peu comme l'esquisse d'un architecte avant de construire une maison – on parle en concepts purs, pas encore en SQL ou en tables MySQL.

Pour l'application de recettes, la structure conceptuelle est simple mais claire. Il y a **trois entités principales** :

L'entité UTILISATEURS représente les comptes des utilisateurs de l'application. Chaque utilisateur a une **identité unique** (un ID), un **nom** qui sert d'identifiant de connexion (et doit être unique parmi tous les utilisateurs), un **mot de passe** pour l'authentification, et un **rôle** déterminant ses permissions (soit "user" pour un utilisateur normal, soit "admin" pour un modérateur). Conceptuellement, un utilisateur est une entité simple avec des attributs simples. Dans la vraie application, il peut y avoir d'autres attributs – email, date de création, date de dernière connexion – mais pour ce projet, ces quatre attributs suffisent.

L'entité RECETTES représente les recettes dans le catalogue. Chaque recette a un **ID unique**, un **titre** lisible (« Pâtes Carbonara »), une **description** optionnelle, une liste **d'ingrédients** (complexe : chacun avec nom, quantité, unité), une liste **d'étapes de préparation** (aussi complexe), une **catégorie** parmi une liste stricte (entrée, plat, dessert, boisson), un **temps de préparation** en minutes, un **nombre de portions**, une **photo** (URL ou chemin fichier), et une **référence à l'utilisateur** qui l'a créée. C'est plus complexe qu'UTILISATEURS, mais chaque attribut correspond à une donnée concrète.

L'entité FAVORIS représente une **relation** entre utilisateurs et recettes. C'est une table de jonction. Conceptuellement, elle relie deux entités : un utilisateur peut marquer plusieurs recettes comme favorites, et une recette peut être marquée comme favorite par plusieurs utilisateurs. C'est une relation **N-N** (plusieurs-à-plusieurs). FAVORIS a peu d'attributs lui-même – juste une référence à l'utilisateur (userId) et une référence à la recette (recetteId). Potentiellement un timestamp de quand l'utilisateur a marqué ce favori (utile pour afficher les favoris les plus récents en premier).

Les relations entre entités sont claires :
Relation 1-N (Un-à-Plusieurs) entre UTILISATEURS et RECETTES : Un utilisateur crée une ou plusieurs recettes. Une recette est créée par exactement un utilisateur. C'est une dépendance naturelle – chaque recette a un créateur.
Relation N-N (Plusieurs-à-Plusieurs) entre UTILISATEURS et RECETTES via FAVORIS : Un utilisateur peut aimer plusieurs recettes. Une recette peut être aimée par plusieurs utilisateurs. Cette relation ne peut pas être représentée par une simple colonne – elle demande une table intermédiaire (FAVORIS).
C'est la beauté du modèle conceptuel – on parle en termes de **concepts métier**, pas en termes techniques. Un développeur non-technicien peut comprendre ce MCD. On n'a pas encore parlé de SQL, de clés primaires, d'indexes, de types de données – c'est abstrait.

2. Modèle Logique (MLD) – Le pont entre concept et SQL

Entre la vision abstraite du MCD et l'implémentation SQL concrète du MPD, il y a une étape intermédiaire trop souvent zappée : le Modèle Logique de Données. Le MLD traduit chaque entité conceptuelle en table, chaque relation en clé étrangère, sans encore se soucier des types de données MySQL exacts ni des options de stockage. C'est l'étape où on décide concrètement : quelle table porte la clé étrangère ? Comment nomme-t-on les colonnes ? Où place-t-on les contraintes d'unicité ?

La traduction suit des règles précises. Chaque entité du MCD devient une table. Chaque relation 1-N se traduit par une clé étrangère placée du côté "N" de la relation – ici, un utilisateur crée plusieurs recettes, donc c'est la table RECETTES qui porte la référence vers UTILISATEURS, jamais l'inverse. Chaque relation N-N, en revanche, ne peut pas se réduire à une simple colonne : elle exige une table de jonction, qui porte les deux clés étrangères des entités reliées. C'est exactement le rôle de la table FAVORIS.

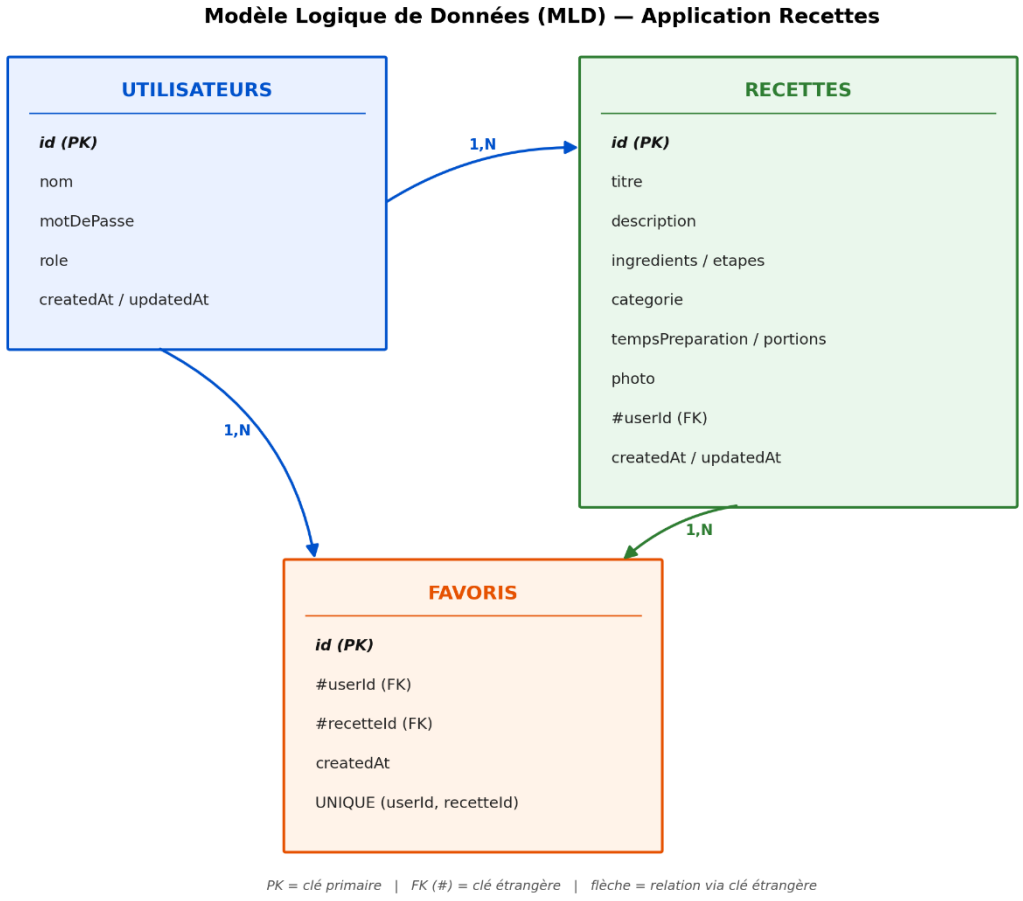
En notation relationnelle standard (clé primaire soulignée, clé étrangère précédée de #), le schéma logique de l'application s'écrit ainsi :

```
UTILISATEURS (id, nom, motDePasse, role, createdAt, updatedAt)

RECETTES (id, titre, description, ingredients, etapes, categorie, tempsPreparation,
portions, photo, #userId, createdAt, updatedAt)

FAVORIS (id, #userId, #recetteId, createdAt)
```

Cette écriture rend visible d'un coup d'œil ce que le MCD laissait encore implicite : RECETTES dépend de UTILISATEURS via #userId, et FAVORIS dépend des deux via #userId et #recetteId, avec la contrainte d'unicité du couple (userId, recetteId) qui garantit qu'un utilisateur ne peut pas favoriser deux fois la même recette. C'est précisément ce schéma logique qui a servi de base directe à l'écriture des instructions CREATE TABLE présentées dans le Modèle Physique ci-après – chaque ligne du MLD correspond presque littéralement à une colonne SQL, à ceci près qu'il reste à choisir les types précis (INT, VARCHAR, JSON, ENUM) et les contraintes de stockage, ce que fait le MPD.



3. Modèle Physique (MPD) – La réalité SQL

Une fois le MCD approuvé, il faut le **traduire en SQL concret**. C'est le Modèle Physique. Ici, on descend au niveau détail – types de données exacts, contraintes, performance.

La table UTILISATEURS se crée ainsi :

```
CREATE TABLE utilisateurs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nom VARCHAR(255) NOT NULL UNIQUE,
  motDePasse VARCHAR(255) NOT NULL,
  role ENUM('user', 'admin') DEFAULT 'user',
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

Détaillons les décisions de design :

- **id INT PRIMARY KEY AUTO_INCREMENT** : L'identifiant unique pour chaque utilisateur. Les nouvelles lignes reçoivent automatiquement un ID incrémenté (1, 2, 3, etc.). C'est performant pour les lookups – chercher par ID c'est ultra-rapide.
- **nom VARCHAR(255) NOT NULL UNIQUE** : Le nom d'utilisateur, maximum 255 caractères. Il est obligatoire (NOT NULL) et unique – pas deux users même nom. Cela force l'unicité au niveau database, pas juste dans le code application (qui peut bugger).
- **motDePasse VARCHAR(255) NOT NULL** : Le password. Mais **JAMAIS en plain text** – toujours hashé avec bcryptjs (10 rounds). Le hash bcryptjs fait ~70 caractères, d'où les 255 alloués généreusement.
- **role ENUM('user','admin') DEFAULT 'user'** : Une énumération stricte. Seules deux valeurs possibles. Si quelqu'un essaie d'insérer 'superadmin' ou n'importe quoi d'autre, MySQL rejette. C'est une validation au niveau database, pas juste application. Le défaut est 'user' – un nouvel utilisateur est normal par défaut, pas admin.
- **createdAt et updatedAt TIMESTAMP** : Audit trail. Quand a-t-on créé ce compte ? Quand l'a-t-on modifié pour la dernière fois ? Utile pour déboguer, pour auditer les changements, pour afficher "Compte créé il y a 2 ans" dans l'interface.

La table RECETTES est plus complexe :

```
CREATE TABLE recettes (
  id INT PRIMARY KEY AUTO_INCREMENT,
  titre VARCHAR(255) NOT NULL,
  description TEXT,
  ingredients JSON NOT NULL,
  etapes JSON NOT NULL,
  categorie ENUM('entree', 'plat', 'dessert', 'boisson') NOT NULL,
  tempsPreparation INT,
  portions INT,
  photo VARCHAR(500),
  userId INT NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (userId) REFERENCES utilisateurs(id) ON DELETE CASCADE,
  INDEX idx_categorie (categorie),
  INDEX idx_userId (userId)
);
```

Détails importants :

- **titre VARCHAR(255)** : Limité à 255 caractères. Assez pour un titre de recette.
- **description TEXT** : Peut être très long (jusqu'à 65k caractères). TEXT c'est pour du contenu potentiellement long.
- **ingredients JSON et etapes JSON** : Les deux sont en JSON, un format structuré flexible. Plutôt que d'avoir une table ingrédients séparée avec 20 colonnes par recette (mauvaise design), on les stocke en JSON. Exemple :
  [

```
    {"nom": "farine", "quantite": 200, "unite": "g"},
    {"nom": "oeufs", "quantite": 3, "unite": ""},
    {"nom": "beurre", "quantite": 100, "unite": "g"}
]
```

MySQL peut interroger du JSON (SELECT * FROM recettes WHERE JSON_CONTAINS(ingredients, '{"nom": "farine"}')) mais c'est flexible.

- **categorie ENUM(...)** : Strict. Seule 4 valeurs. Pas de "Soupe" ou "Snack" qui n'ont pas été prévus.
- **tempsPreparation INT** : En minutes. Simple entier.
- **userId INT NOT NULL** : Clé étrangère pointant vers utilisateurs(id). Chaque recette doit avoir un créateur.
- **FOREIGN KEY (userId) REFERENCES utilisateurs(id) ON DELETE CASCADE** : C'est la vraie puissance. Si on supprime un utilisateur de la table utilisateurs, MySQL supprime automatiquement TOUTES ses recettes. Pas d'orphelins. La contrainte ON DELETE CASCADE assure la cohérence.
- **INDEX idx_categorie et INDEX idx_userId** : Les indexes accélèrent les recherches. Si on interroge souvent par catégorie (« affiche-moi toutes les recettes de type 'plat' »), un index sur categorie rend ça ultra-rapide. Pareil pour userId – chercher toutes les recettes d'un user est courant.

La table FAVORIS relie les deux :

```
CREATE TABLE favoris (
  id INT PRIMARY KEY AUTO_INCREMENT,
  userId INT NOT NULL,
  recetteId INT NOT NULL,
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (userId) REFERENCES utilisateurs(id) ON DELETE CASCADE,
  FOREIGN KEY (recetteId) REFERENCES recettes(id) ON DELETE CASCADE,
  UNIQUE KEY uk_user_recipe (userId, recetteId),
  INDEX idx_userId (userId),
  INDEX idx_recetteId (recetteId)
);
```

Détails clés :

- **userId et recetteId** : Deux clés étrangères. Chaque favori doit référencer un user et une recette valides.
- **UNIQUE KEY uk_user_recipe (userId, recetteId)** : C'est la vraie brillance. Le couple (userId, recetteId) doit être unique. Un utilisateur ne peut marquer une recette qu'une seule fois en favoris. Si quelqu'un tente une deuxième fois, MySQL rejette avec une erreur UNIQUE constraint.
- **CASCADE ON DELETE** sur les deux FK : Si on supprime un user, tous ses favoris partent. Si on supprime une recette, ses entrées favoris partent. Intégrité garantie.
- **Indexes sur userId et recetteId** : Optimisation. Chercher "tous les favoris de l'user 5" est rapide.

Attention à un détail crucial : Les trois tables ne contiennent **aucune redondance**. Le nom de l'utilisateur qui crée une recette ? On stocke juste son ID (userId). Si on veut le nom complet pour afficher, on fait une JOIN avec la table utilisateurs. Cela signifie : si le user change son nom, il change une seule fois dans utilisateurs, et partout où il apparaît (via la relation) le change automatiquement. C'est la force du modèle relationnel.

4. Normalisation 3NF – Pas de redondance

La "normalisation" est un ensemble de règles qui garantissent qu'une structure de base de données est bien pensée, sans redondance nuisible, sans anomalies de mise à jour.

Première Forme Normale (1NF) – Atomicité : Chaque colonne contient une **valeur unique, indivisible** (atomique). Pas de multi-valeurs. La mauvaise approche : avoir une colonne "ingredients" contenant du texte comme "farine (200g), oeufs (3), beurre (100g)". C'est une seule chaîne – pas atomique. On ne peut pas chercher "recettes avec farine". Le bon approche : JSON ou une table séparée. Nos recettes respectent 1NF – chaque colonne une valeur unique.

Deuxième Forme Normale (2NF) – Dépendances fonctionnelles : Suppose 1NF. Ensuite, **tous les attributs non-clé dépendent de la clé primaire entière, pas d'une partie**. Exemple de violation 2NF : avoir une table "recipe_creators" avec colonnes (recetteId, userId, userNom). Le userNom dépend de userId, pas de recetteId. Si on change le nom du user, il faudrait mettre à jour partout – anomalie. La solution : garder juste (recetteId, userId)

et faire une JOIN avec utilisateurs pour le nom. Nos tables respectent 2NF – chaque attribut dépend de la clé entière.

Troisième Forme Normale (3NF) – Pas de dépendances transitives : Suppose 2NF. Ensuite, **aucun attribut non-clé ne dépend d'un autre attribut non-clé**. Exemple de violation 3NF : table "orders" avec colonnes (orderId, customerId, customerCountry, countryTaxRate). Le countryTaxRate dépend de customerCountry, qui dépend de customerId. C'est transitif. La solution : créer une table "countries" et faire une référence. Nos tables respectent 3NF – pas de telles dépendances.

Pourquoi 3NF ? Parce que ça élimine les **anomalies de mise à jour**. Si les données sont bien normalisées :

- **Anomalies d'insertion** : Impossible d'insérer données incohérentes (via les contraintes FK)
- **Anomalies de suppression** : Pas de perte accidentelle d'infos quand on supprime une ligne (via CASCADE on DELETE géré consciemment)
- **Anomalies de modification** : Une modification de donnée se fait une seule fois, se propage partout via les relations (pas de copies divergentes)

Nos trois tables (utilisateurs, recettes, favoris) respectent strictement 3NF. Elles ne contiennent aucune redondance inutile.

5. Intégrité référentielle – Les contraintes

L'intégrité référentielle signifie que **toutes les références entre tables pointent vers des données valides**. Une recette ne peut pas référencer un utilisateur inexistant. Un favori ne peut pas référencer une recette supprimée. MySQL impose cela via les **Foreign Key constraints** avec options **ON DELETE** et **ON UPDATE**.

Scenario 1 : Suppression d'un utilisateur

Un user crée 5 recettes, et 10 autres users marquent 3 de ses recettes en favoris. Si on supprime ce user, que se passe-t-il ?
Avec ON DELETE CASCADE sur la FK userId dans recettes :

- Les 5 recettes créées par ce user sont **automatiquement supprimées**
- Quand ces 5 recettes sont supprimées, ON DELETE CASCADE sur la FK recetteId dans favoris déclenche
- Les entrées favoris concernant ces recettes sont **automatiquement supprimées**
- Résultat : **Zéro orphelin. Cohérence garantie.**

Sans CASCADE (mauvaise approche) :

- La suppression du user échouerait (contrainte FK empêcherait)
- Ou ses recettes resteraient orphelines, pointant vers un user inexistant
- Anomalie.

Scenario 2 : Suppression d'une recette

Un user supprime une recette qu'il a créée. 15 autres users l'ont marquée en favori. Avec ON DELETE CASCADE sur recetteId dans favoris :

- La recette est supprimée
- Automatiquement, les 15 entrées favoris la concernant sont **supprimées**
- Les 15 users voient le favori disparaître de leur liste
- C'est le comportement attendu – une recette supprimée n'apparaît nulle part

Scenario 3 : Modification d'ID utilisateur

C'est rare (on ne change pas les ID en général), mais conceptuellement, avec ON UPDATE CASCADE :

- Si l'ID d'un user change (mauvaise idée en production, mais techniquement possible), toutes les FK qui le référencent se mettent à jour automatiquement
- Zéro perte cohérence

Ces contraintes ne sont pas juste "nice to have" – c'est de la vraie sécurité. Elles empêchent les bugs au niveau database. Un programmer bug qui tente de supprimer un user sans gérer ses recettes ? MySQL le bloque. Un bug qui insère une recette avec userId inexistant ? MySQL le rejette.

6. Dictionnaire des données

Pour complétude, voici un dictionnaire simple documentant chaque colonne :

Table UTILISATEURS

- **id** : Type INT, clé primaire, auto-incrémentée. Identifiant unique.
- **nom** : Type VARCHAR(255), unique, non-null. Nom d'utilisateur pour connexion. Contrainte UNIQUE.
- **motDePasse** : Type VARCHAR(255), non-null. Hash bcryptjs 10 rounds. Jamais plain text.
- **role** : Type ENUM('user', 'admin'), défaut 'user'. Détermine permissions. Strict – pas d'autres valeurs.
- **createdAt** : Type TIMESTAMP, défaut NOW(). Quand le compte a-t-il été créé. Audit.
- **updatedAt** : Type TIMESTAMP, défaut NOW(), auto-update. Dernière modification. Audit.

Table RECETTES

- **id** : Type INT, clé primaire, auto-incrémentée. Identifiant unique.
- **titre** : Type VARCHAR(255), non-null. Nom de la recette (ex: "Pâtes Carbonara").
- **description** : Type TEXT, optionnel. Description longue de la recette.
- **ingredients** : Type JSON, non-null. Array JSON d'objets {nom, quantite, unite}.
- **etapes** : Type JSON, non-null. Array JSON d'objets {numero, instruction, duree}.
- **categorie** : Type ENUM('entree', 'plat', 'dessert', 'boisson'), non-null. Strict validation.
- **tempsPreparation** : Type INT, optionnel. Temps en minutes.
- **portions** : Type INT, optionnel. Nombre de portions.
- **photo** : Type VARCHAR(500), optionnel. URL ou chemin vers image.
- **userId** : Type INT, non-null, clé étrangère vers utilisateurs(id). Créateur de la recette.
- **createdAt** : Type TIMESTAMP, défaut NOW(). Audit.
- **updatedAt** : Type TIMESTAMP, défaut NOW(), auto-update. Audit.

Table FAVORIS

- **id** : Type INT, clé primaire, auto-incrémentée. Identifiant unique.
- **userId** : Type INT, non-null, clé étrangère vers utilisateurs(id). Utilisateur qui a mis en favoris.
- **recetteId** : Type INT, non-null, clé étrangère vers recettes(id). Recette mise en favoris.
- **createdAt** : Type TIMESTAMP, défaut NOW(). Quand l'utilisateur a marqué ce favori.

Contraintes :

- Primary Key : id
- Foreign Keys : userId → utilisateurs(id) ON DELETE CASCADE, recetteId → recettes(id) ON DELETE CASCADE
- Unique : (userId, recetteId) – un user ne peut favoriter une recette qu'une fois

7. Exemple de données concrètes

Pour vraiment visualiser, voici quelques données exemples :

Utilisateurs :

ID	NOM	MOTDEPASSE	ROLE	CREATEDAT
1	admin	$2a$10$...hash bcryptjs...	admin	2025-01-01
2	jean	$2a$10$...autre hash...	user	2025-01-05
3	marie	$2a$10$...autre hash...	user	2025-01-10

Recettes :

ID	TITRE	CATEGORIE	TEMPSPREPARATION	PORTIONS	USERID	CREATEDAT
1	Pâtes Carbonara	plat	20	4	1	2025-01-02
2	Soupe à l'oignon	entree	15	2	1	2025-01-03
3	Brownies Chocolat	dessert	30	6	2	2025-01-06
4	Salade Grecque	entree	10	3	3	2025-01-11

Favoris :

ID	USERID	RECETTEID	CREATEDAT	COMMENTAIRE
1	2	1	2025-01-06	Jean a mis en favoris les Pâtes Carbonara
2	3	1	2025-01-12	Marie aussi aime les Pâtes Carbonara
3	2	3	2025-01-07	Jean aime les Brownies
4	3	4	2025-01-12	Marie aime sa propre recette Salade Grecque

Si on supprime Jean (userId 2) :

- Les recettes créées par Jean ? Supprimées (Pâtes Carbonara, Soupe – id 1, 2)
- Les entrées favoris impliquant Jean ou ses recettes ? Supprimées (id 1, 3)
- Marie (userId 3) garde ses recettes et ses favoris – elle n'est pas affectée

Voilà la puissance de la conception et des contraintes appropriées.

V. DÉVELOPPEMENT BACKEND

Le backend est le cœur invisible de l'application : là où les données sont validées, la logique métier appliquée, et la communication avec la base de données assurée. Il repose sur Node.js, Express, et Sequelize comme ORM vers MySQL, organisé selon le pattern RCSM (Route → Controller → Service → Model) déjà présenté dans la section Architecture.

1. Organisation des fichiers

Le backend est structuré en dossiers correspondant chacun à une couche du pattern RCSM : `routes/` définit les endpoints et leurs middlewares associés, `controllers/` reçoit les requêtes et orchestre la réponse, services/ porte la logique métier réelle, `models/` définit les entités Sequelize et leurs associations, et `middlewares/` regroupe les fonctions d'authentification et d'autorisation. Le flux d'une requête suit systématiquement cet ordre : route → controller → service → model → base de données, puis la réponse remonte cette même chaîne jusqu'au client.

2. Modèles Sequelize

Chaque modèle (`Utilisateur`, `Recette`, `Favori`) définit sa structure de colonnes, ses contraintes (unicité, longueur, format) et ses validations directement au niveau du schéma, ce qui garantit qu'une donnée invalide ne peut pas être persistée même si elle a échoué à être filtrée en amont. Les associations (un utilisateur possède plusieurs recettes, une relation many-to-many entre utilisateurs et recettes via la table de jonction `Favori`) sont déclarées dans `models/index.js`, ce qui permet à Sequelize de générer automatiquement les jointures nécessaires lors des requêtes. Le code complet des modèles est disponible sur le dépôt GitHub.

3. Services métier

Les services encapsulent la logique que les controllers ne devraient pas porter eux-mêmes : validations métier complexes, orchestration de plusieurs opérations, journalisation. Le service d'authentification, par exemple, vérifie l'existence de l'utilisateur, compare le mot de passe fourni au hash stocké via bcryptjs, puis génère un token JWT valide 24 heures contenant l'identifiant, le nom et le rôle de l'utilisateur. Le service des recettes gère quant à lui le catalogue paginé et filtrable par catégorie, ainsi que les validations métier à la création (présence d'au moins un ingrédient et une étape) avant de déléguer la persistance au modèle Sequelize.

4. Middleware d'authentification

Deux middlewares protègent les routes sensibles : `isAuthenticated` vérifie la présence et la validité du token JWT transmis dans l'en-tête `Authorization`, et attache l'utilisateur décodé à la requête ; `isAdmin`, exécuté après le précédent, vérifie que le rôle de l'utilisateur autorise l'action demandée.

```
const isAuthenticated = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token manquant' });
  }
  try {
    req.user = jwt.verify(authHeader.slice(7), process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Token invalide ou expiré' });
  }
};
```

Ce middleware est appliqué sélectivement sur les routes qui le nécessitent (création, modification, suppression de recette, gestion des favoris), tandis que les routes de consultation publique du catalogue restent accessibles sans authentification.

5. Routes et point d'entrée

Les fichiers de routes déclarent chaque endpoint, associent les middlewares nécessaires, puis délèguent au controller correspondant — par exemple `router.post('/recettes', isAuthenticated, recetteController.create)`. Le point d'entrée `index.js` initialise l'application Express, configure les middlewares globaux (CORS, parsing JSON, gestion des uploads via Multer), connecte l'ensemble des routes, établit la connexion à la base de données, puis démarre le serveur sur le port configuré.

VI. DÉVELOPPEMENT FRONTEND REACT

Le frontend est la partie visible de l'application : ce que l'utilisateur voit, touche et utilise au quotidien. Il s'agit d'une application React 19, organisée en composants réutilisables, connectée au backend via une API REST, et pensée pour rester fluide et accessible.

1. Architecture globale React – Pensée en composants

React invite à raisonner en briques réutilisables plutôt qu'en pages entières. Le projet est structuré en dossiers clairs : `components/` pour les éléments réutilisables (carte recette, formulaires, barre de navigation, bouton favoris...), `pages/` pour les vues associées aux routes, `context/` pour l'état global d'authentification, `services/` pour la communication avec l'API, `hooks/` pour la logique réutilisable, et `styles/` pour les feuilles SCSS. Cette séparation facilite la maintenance : chaque fichier a une responsabilité unique.

Le cycle de vie d'un composant suit un schéma constant : réception des props, initialisation du state via `useState`, déclenchement des effets de bord via `useEffect` (appels API, abonnements), rendu JSX, puis mise à jour du state suite aux interactions utilisateur — ce qui déclenche un nouveau rendu ciblé. C'est ce cycle prévisible qui rend React efficace : seuls les éléments modifiés sont recalculés.

2. Context API – Authentification globale

L'état de connexion (utilisateur courant, token, droits) est nécessaire dans presque tous les composants. Plutôt que de le faire transiter par de nombreux niveaux de props, l'application utilise le Context API de React : un `AuthProvider` englobe toute l'application, vérifie au démarrage la validité d'un éventuel token stocké, et expose `user`, `login()`, `logout()`, ainsi que des booléens dérivés (`isAuthenticated`, `isAdmin`). N'importe quel composant peut ensuite consommer ce contexte via `useContext(AuthContext)` sans avoir à recevoir ces informations par props — c'est le cas par exemple de la barre de navigation, qui affiche conditionnellement le nom de l'utilisateur connecté ou un lien de connexion.

3. Composants réutilisables – Briques de l'UI

Les composants principaux — carte recette, liste/grille de recettes, formulaire de connexion, formulaire de création/édition de recette, bouton favoris, spinner de chargement — suivent tous la même logique : ils reçoivent des données via props, gèrent un minimum d'état local si nécessaire, et restent indépendants du contexte dans lequel ils sont utilisés. Cette indépendance permet par exemple de réutiliser `RecipeCard` aussi bien dans le catalogue que dans la page des favoris, sans dupliquer de code. Le code complet de ces composants est disponible sur le dépôt GitHub.

4. Gestion des formulaires et validations

Les formulaires (connexion, création de recette) utilisent des composants contrôlés : chaque champ est lié à une valeur de state, mise à jour à chaque saisie. Une validation côté client (champs requis, formats, longueurs) est effectuée avant l'envoi, en complément de la validation côté serveur qui reste la garantie de sécurité définitive. Les erreurs sont affichées au plus près du champ concerné pour guider l'utilisateur.

5. Communication avec l'API

Une instance Axios centralisée (`services/api.js`) configure l'URL de base et injecte automatiquement le token d'authentification dans les en-têtes des requêtes. Les appels sont ensuite organisés par domaine (`authService`, `recetteService`, `favoriService`), ce qui évite de disperser des appels `fetch` bruts dans les composants. La récupération de données au chargement d'une page suit un schéma classique combinant `useState` et `useEffect` :

```
useEffect(() => {
  const fetchRecettes = async () => {
    try {
      const data = await recetteService.getAll();
      setRecettes(data);
    } catch (err) {
      setError("Impossible de charger les recettes");
    } finally {
      setLoading(false);
    }
  };
  fetchRecettes();
}, []);
```

Le tableau de dépendances vide indique que l'effet ne s'exécute qu'au montage du composant, ce qui correspond au besoin d'un chargement initial unique.

6. Navigation et style

La navigation entre les pages est gérée par React Router, avec une route protégée (`ProtectedRoute`) qui redirige vers la page de connexion tout utilisateur non authentifié tentant d'accéder à une page réservée (favoris, administration). Le style visuel repose sur SCSS, organisé en variables globales (couleurs, espacements) et en fichiers dédiés par composant ou par page, garantissant une cohérence visuelle sans dupliquer les règles CSS.

VII. SÉCURITÉ & AUTHENTIFICATION

La sécurité a été traitée comme un impératif dès la conception, non comme un ajout final. Cette section présente la couverture des risques OWASP Top 10 et le détail des mécanismes de protection réellement implémentés.

1. Couverture OWASP Top 10 2021

Risque	Protection appliquée
A01 – Broken Access Control	Middleware isAuthenticated + isAdmin, vérification systématique de la propriété des ressources avant modification/suppression
A02 – Cryptographic Failures	Mots de passe hachés avec bcryptjs, secrets JWT et identifiants BDD hors du code source (.env)
A03 – Injection	Requêtes paramétrées via Sequelize (jamais de SQL concaténé), validation stricte des entrées
A04 – Insecure Design	Séparation des responsabilités (RCSM), validations dupliquées client/serveur
A05 – Security Misconfiguration	Variables sensibles hors dépôt, en-têtes de sécurité (Helmet), messages d'erreur génériques en production
A06 – Vulnerable Components	npm audit exécuté régulièrement, dépendances maintenues à jour
A07 – Authentication Failures	JWT à durée de vie limitée (24h), mots de passe soumis à des règles de complexité
A08 – Data Integrity Failures	Validation des schémas Sequelize, contrôle des types et formats à chaque écriture
A09 – Logging Failures	Journalisation des actions sensibles (création/suppression de recettes, tentatives de connexion)
A10 – Known Vulnerabilities	Couvert par la veille sur les dépendances (A06)

2. Authentification par JWT

Le choix du JWT (JSON Web Token) plutôt que des sessions serveur classiques se justifie par la nature stateless de l'API : aucune donnée de session n'est stockée côté serveur, ce qui simplifie la scalabilité. À la connexion, le service d'authentification génère un token signé contenant l'identifiant, le nom et le rôle de l'utilisateur, avec une expiration fixée à 24 heures — un compromis entre confort d'utilisation et réduction de la fenêtre d'exposition en cas de vol de token. Ce token est ensuite exigé dans l'en-tête `Authorization` de chaque requête protégée, vérifié par le middleware `isAuthenticated` présenté dans la section Backend.

3. Validation des données côté serveur

Chaque endpoint qui reçoit des données utilisateur les valide avant tout traitement : présence des champs requis, formats attendus (email, longueur de mot de passe), typage cohérent avec le schéma Sequelize. Cette validation côté serveur est la garantie de sécurité réelle — la validation côté client (section Frontend) n'est qu'un confort d'expérience utilisateur, jamais une protection suffisante en soi, puisqu'un attaquant peut toujours contourner le frontend et interroger l'API directement.

4. Hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. Un hook Sequelize intercepte la sauvegarde d'un utilisateur et hache le mot de passe avec bcryptjs avant écriture en base :

```
Utilisateur.beforeCreate(async (user) => {
  user.motDePasse = await bcrypt.hash(user.motDePasse, 10);
});
```

Le facteur de coût (10) équilibre robustesse contre les attaques par force brute et temps de calcul acceptable. À la connexion, bcrypt.compare() compare le mot de passe fourni au hash stocké, sans jamais avoir besoin de déchiffrer ce dernier — bcrypt est un hachage à sens unique, pas un chiffrement réversible.

5. Autres protections

L'application configure également CORS pour restreindre les origines autorisées à appeler l'API, utilise Helmet pour ajouter des en-têtes HTTP de sécurité standard, et limite la taille des fichiers uploadés via Multer afin d'éviter les abus. Une checklist de sécurité a été suivie avant chaque mise en production, couvrant notamment l'absence de secrets dans le dépôt Git, la désactivation des logs verbeux, et la vérification npm audit sans vulnérabilité critique restante.

VIII. TESTS FONCTIONNELS & QUALITÉ

Une application sans tests est fragile : à chaque modification, on risque de casser une fonctionnalité existante sans s'en rendre compte. Les tests garantissent que le comportement attendu reste vérifié, aujourd'hui comme après chaque évolution du code.

1. Stratégie de test

Le projet suit la pyramide de tests classique : une large base de tests unitaires (rapides, isolés, testant une fonction ou un service indépendamment de ses dépendances, avec des données mockées), un niveau intermédiaire de tests d'intégration qui s'appuient sur une base de données de test réinitialisée avant chaque exécution et peuplée avec le même jeu de données que l'application — les 21 recettes de test réparties sur les quatre catégories (5 entrées, 8 plats, 6 desserts, 2 boissons) — et quelques tests de bout en bout sur les parcours utilisateurs les plus critiques (connexion, création de recette, gestion des favoris). Cette répartition permet d'obtenir une couverture large sans payer le coût élevé de tests end-to-end sur chaque cas, tout en gardant des résultats attendus vérifiables sur des données réelles plutôt qu'arbitraires.

```
-- ============================================
-- SCRIPT D'INSERTION DE 20 RECETTES
-- ============================================

INSERT INTO `recettes` (`nom`, `description`, `ingredients`, `etapes`, `auteur`, `categorie`, `temps`, `portions`, `photo`, `createdAt`, `updatedAt`) VALUES

-- ENTRÉES (5)
('Salade Niçoise', 'Salade fraîche avec thon, oeufs et olives', 'Salade laitue\nThon en conserve\nOeufs durs\nTomates\nOlives noires\nOignons rouges\nAnchois', 'Laver la laitue\nCouper les tomates\nCuire les oeufs\nMélanger tous les ingrédients\nAjouter la vinaigrette', 'Grand-mère', 'entree', '15 min', 4, NULL, NOW(), NOW()),

('Soupe à l\'Oignon', 'Soupe gratinée française classique', 'Oignons\nBouillon de boeuf\nBeurre\nPain\nFromage Gruyère\nSel et poivre', 'Émincer les oignons\nLes faire revenir au beurre\nAjouter le bouillon\nLaisser cuire 30 min\nMettre le pain et fromage sur le dessus\nGratiner au four', 'Maman', 'entree', '45 min', 4, NULL, NOW(), NOW()),

('Œufs Mayonnaise', 'Oeufs durs avec mayonnaise maison', 'Oeufs\nJaunes d\'oeufs\nHuile\nVinaigre blanc\nMoutarde\nSel et poivre', 'Cuire les oeufs\nFaire la mayonnaise\nCouper les oeufs en deux\nGarnir de mayonnaise', 'Tante Marie', 'entree', '20 min', 4, NULL, NOW(), NOW()),

('Crevettes à l\'Ail', 'Crevettes sautées à l\'ail et persil', 'Crevettes fraîches\nAil\nPersil frais\nHuile d\'olive\nCitron\nSel et poivre', 'Nettoyer les crevettes\nFaire revenir l\'ail\nAjouter les crevettes\nCuire 3-4 minutes\nAjouter le persil et citron', 'Chef Pierre', 'entree', '12 min', 4, NULL, NOW(), NOW()),
```

```
-- PLATS (8)
('Coq au Vin', 'Coq braisé au vin rouge', 'Coq\nVin rouge\nLardons\nOignons\nChampignons\nGarni\nBouillon', 'Faire revenir les lardons\nAjouter le coq doré\nVerser le vin\nAjouter les légumes\nCuire 2h à couvert', 'Grand-mère', 'plat', '150 min', 6, NULL, NOW(), NOW()),

('Pâtes Carbonara', 'Pâtes à la crème et bacon', 'Pâtes\nOeufs\nBacon\nParmesan\nSel et poivre', 'Faire bouillir l\'eau\nCuire les pâtes\nFaire revenir le bacon\nMélanger les oeufs\nCombiner tout\nAjouter le fromage', 'Maman', 'plat', '20 min', 4, NULL, NOW(), NOW()),

('Boeuf Bourguignon', 'Ragoût de boeuf au vin', 'Boeuf\nVin rouge\nCarottes\nOignons\nChampignons\nBouillon\nTomate', 'Faire revenir le boeuf\nAjouter les légumes\nVerser le vin et bouillon\nCuire 3h à couvert\nGarnir de persil', 'Tante Marie', 'plat', '200 min', 6, NULL, NOW(), NOW()),

('Risotto aux Champignons', 'Riz crémeux aux champignons', 'Riz Arborio\nChampignons\nOignon\nBouillon chaud\nVin blanc\nParmesan\nBeurre', 'Faire revenir l\'oignon\nAjouter le riz\nVerser le vin\nAjouter le bouillon progressivement\nMélanger régulièrement\nAjouter beurre et fromage', 'Chef', 'plat', '30 min', 4, NULL, NOW(), NOW()),

('Poulet Rôti', 'Poulet rôti simple et succulent', 'Poulet entier\nHerbes de Provence\nBeurre\nAil\nCitron\nSel et poivre', 'Préparer le poulet\nLe badigeonner de beurre\nAjouter les herbes et ail\nRôtir 1h30\nLaisser reposer 10 min', 'Grand-mère', 'plat', '120 min', 6, NULL, NOW(), NOW()),

('Moules Marinières', 'Moules à la crème et vin blanc', 'Moules\nVin blanc\nEchalotes\nBeurre\nCrème fraîche\nPersil', 'Nettoyer les moules\nFaire revenir les échalotes\nAjouter les moules\nVerser le vin\nCuire jusqu\'à ouverture\nAjouter la crème', 'Chef Bretagne', 'plat', '25 min', 4, NULL, NOW(), NOW()),

('Steak Frites', 'Entrecôte avec frites croustillantes', 'Entrecôte\nPommes de terre\nHuile\nSel\nPoivre\nBeurre', 'Peler les pommes de terre\nCouper en bâtons\nFrire 2 fois\nCuire le steak\nAssaisonner', 'Papa', 'plat', '45 min', 2, NULL, NOW(), NOW()),

('Cassoulet Toulousain', 'Ragoût de haricots et confit', 'Haricots blancs\nConfit de canard\nSaucisse\nOignons\nAil\nBouillon', 'Tremper les haricots\nLes cuire\nAjouter le confit et saucisse\nCuire 1h\nLaisser croûter au four', 'Grand-mère du Sud', 'plat', '240 min', 8, NULL, NOW(), NOW()),
```

```
-- DESSERTS (5)
('Crème Brûlée', 'Crème anglaise avec sucre caramélisé', 'Lait entier\nJaunes d\'oeufs\nSucre\nVanille\nSel', 'Faire bouillir le lait\nMélanger jaunes et sucre\nTemp érature basse\nCuire au bain-marie\nRefroidir et caraméliser', 'Chef Pâtissier', 'dessert', '45 min', 4, NULL, NOW(), NOW()),

('Tarte aux Pommes', 'Tarte classique aux pommes caramélisées', 'Pâte brisée\nPommes\nSucre\nBeurre\nCanelle\nOeufs', 'Préparer la pâte\nCuire à blanc\nPréparer les pommes\nMettres sur la pâte\nCuire 35 min', 'Grand-mère', 'dessert', '90 min', 6, NULL, NOW(), NOW()),

('Mousse au Chocolat', 'Mousse légère et aérée', 'Chocolat noir\nOeufs\nSucre\nCrème fraîche\nSel', 'Faire fondre le chocolat\nMélanger jaunes et sucre\nMonter les blancs\nMélanger délicatement\nRefroidir 2h', 'Chef', 'dessert', '30 min', 4, NULL, NOW(), NOW()),

('Mille-feuille', 'Pâte feuilletée avec crème', 'Pâte feuilletée\nCrème pâtissière\nSucre glace\nChocolat', 'Cuire la pâte feuilletée\nPréparer la crème\nAssembler les couches\nGlacer le dessus\nDécorer', 'Pâtissier', 'dessert', '60 min', 6, NULL, NOW(), NOW()),

('Île Flottante', 'Meringue sur crème anglaise', 'Oeufs\nSucre\nLait\nVanille', 'Monter les blancs\nFormer les meringues\nLes pocher\nPréparer la crème anglaise\nDresser', 'Grand-mère', 'dessert', '40 min', 4, NULL, NOW(), NOW()),

-- BOISSONS (2)
('Vin Chaud', 'Vin rouge épicé et chaud', 'Vin rouge\nCannelle\nClous de girofle\nAnis étoilé\nOrange\nMiel', 'Verser le vin\nAjouter les épices\nChauffer sans bouillir\nLaisser infuser 10 min\nAjouter le miel', 'Grand-mère', 'boisson', '15 min', 4, NULL, NOW(), NOW()),

('Sangria Espagnole', 'Vin blanc avec fruits frais', 'Vin blanc\nOrange\nCitron\nFraises\nMelon\nSucre\nCanelle', 'Couper les fruits\nMélanger avec le vin\nAjouter le sucre\nLaisser reposer 2h\nServir frais', 'Tante Maria', 'boisson', '130 min', 8, NULL, NOW(), NOW());
```

Les tests sont écrits après le développement de chaque fonctionnalité plutôt qu'en TDD strict, mais de façon systématique : aucune fonctionnalité n'est considérée terminée sans son jeu de tests associé.

2. Outillage

Les tests reposent sur Jest comme framework principal (exécution, assertions, mocks) et Supertest pour simuler des requêtes HTTP sur l'API sans avoir à démarrer un serveur réel. Les tests unitaires mockent les dépendances externes (modèles Sequelize, bcryptjs, jsonwebtoken) afin d'isoler strictement la fonction testée, selon le schéma Arrange / Act / Assert : préparation du scénario, exécution du code, puis vérification du résultat.

es tests d'intégration, eux, n'utilisent aucun mock sur la couche base de données : un script de seed exécuté avant la suite de tests recrée les tables et les repeuple avec les 21 recettes de test, garantissant que chaque exécution part d'un état identique et connu.

Un exemple représentatif — le test du service d'authentification — illustre l'approche unitaire :

```
test('devrait retourner token et user quand credentials valides', async () => {
 Utilisateur.findOne.mockResolvedValue(mockUser);
 bcrypt.compare.mockResolvedValue(true);
 jwt.sign.mockReturnValue('fake-jwt-token');
 const result = await authService.authenticate('admin', 'password123');
 expect(result).toEqual({
 token: 'fake-jwt-token',
 user: { id: 1, nom: 'admin', role: 'admin' }
 });
});
```

3. Jeu d'essai — filtrage du catalogue par catégorie

Pour illustrer concrètement le comportement attendu sur une fonctionnalité représentative, voici le jeu d'essai utilisé pour tester le filtrage du catalogue par catégorie, exécuté contre le jeu de données de 21 recettes :

Données en entrée : requête `GET /api/recettes?categorie=dessert` sur la base de test peuplée par le seed des 21 recettes, dont 6 recettes appartiennent à la catégorie "dessert".

Données attendues : réponse `HTTP 200`, tableau JSON de 6 éléments, chacun avec categorie: "dessert".

Données obtenues : réponse `HTTP 200`, tableau JSON de 6 éléments, tous avec categorie: "dessert" — conforme à l'attendu, aucun écart constaté.

Ce même principe (jeu de données connu → résultat attendu calculé à l'avance → comparaison au résultat obtenu) est appliqué aux tests d'intégration sur l'authentification et sur la gestion des favoris, en s'appuyant à chaque fois sur les mêmes 21 recettes et les 3 comptes utilisateurs de test.

4. Couverture et intégration continue

Les tests sont exécutés via npm test, avec une option --coverage générant un rapport détaillé par fichier (instructions, branches, fonctions, lignes couvertes). L'objectif visé est d'au moins 80 % de couverture sur les composants critiques (authentification, gestion des recettes, favoris), un seuil jugé pertinent entre effort et bénéfice.

Un pipeline CI/CD (GitHub Actions) exécute automatiquement l'ensemble des tests à chaque push ou pull request : si un test échoue, la fusion du code est bloquée. Cette barrière qualité empêche qu'une régression non détectée n'atteigne la branche principale.

5. Ce que les tests permettent de détecter

Les tests unitaires révèlent des erreurs de logique pure — un calcul de total oubliant de multiplier par une quantité, par exemple. Les tests d'intégration détectent des problèmes d'enchaînement entre modules, comme une opération non transactionnelle qui laisserait les données dans un état incohérent en cas d'erreur en cours de traitement. Les tests de bout en bout, enfin, révèlent des problèmes d'expérience utilisateur qu'un test unitaire ne verrait pas, comme un bouton visuellement présent mais non réellement connecté à son action.

6. Bonnes pratiques appliquées

Trois principes guident l'écriture des tests dans ce projet : un test ne vérifie qu'un seul comportement à la fois (plutôt que d'empiler plusieurs assertions non liées dans un même test) ; chaque test est indépendant et peut s'exécuter seul, sans dépendre de l'ordre d'exécution des autres ; et les noms de tests décrivent précisément le

comportement attendu (devrait rejeter ajout favori si utilisateur non authentifié plutôt que test 1), ce qui rend les rapports de test directement lisibles comme documentation.

IX. ACCESSIBILITÉ & WCAG 2.1

L'accessibilité vise à rendre l'application utilisable par tous, quelle que soit la situation de handicap ou le contexte d'usage. Elle est aussi une obligation légale pour les services numériques en France et dans l'UE (directive 2016/2102, niveau WCAG 2.1 AA), et bénéficie en pratique à l'ensemble des utilisateurs — un contraste fort profite autant aux malvoyants qu'à une lecture en plein soleil, une navigation clavier fluide profite autant aux personnes sans souris qu'aux utilisateurs avancés.

1. Les quatre principes WCAG (POUR)

La norme s'organise en quatre piliers. **Perceptible** : chaque image dispose d'un texte alternatif, le contraste texte/fond respecte un ratio minimal de 4.5:1, aucune information n'est transmise par la couleur seule. **Opérable** : l'intégralité des éléments interactifs est accessible au clavier, sans piège de focus, avec un ordre de tabulation logique. **Compréhensible** : le langage reste simple, la navigation cohérente d'une page à l'autre, et les erreurs de formulaire sont signalées clairement avant validation. **Robuste** : le HTML sémantique est utilisé partout où c'est pertinent, complété par des attributs ARIA lorsque des composants JavaScript dynamiques le nécessitent.

2. Mise en œuvre côté frontend

L'application privilégie systématiquement les éléments HTML sémantiques (`<header>`, `<nav>`, `<main>`, `<button>`) plutôt que des `<div>` génériques avec gestionnaires de clic, ce qui donne gratuitement aux lecteurs d'écran la structure et le rôle de chaque élément. Les formulaires associent explicitement chaque champ à son label, exposent les erreurs via `aria-describedby`, et signalent les champs invalides avec `aria-invalid` :

```
<label htmlFor="email">Email</label>
<input
  id="email"
  type="email"
  aria-invalid={!!emailError}
  aria-describedby="email-error"
/>
{emailError && <span id="email-error" role="alert">{emailError}</span>}
```

Ce schéma garantit qu'un lecteur d'écran annonce l'erreur au moment où elle apparaît, sans que l'utilisateur ait à la chercher visuellement.

3. Mise en œuvre côté backend

Le backend contribue à l'accessibilité indirectement : les messages d'erreur retournés par l'API sont rédigés en langage clair (plutôt que des codes techniques), ce qui permet au frontend de les afficher directement de façon compréhensible, et les métadonnées des recettes (textes alternatifs d'images notamment) sont stockées et systématiquement renvoyées pour que le frontend puisse les restituer.

4. Tests et vérification

La navigation complète de l'application a été testée au clavier seul (sans souris), sur l'ensemble des parcours critiques (connexion, création de recette, gestion des favoris). Les contrastes ont été vérifiés à l'aide d'outils dédiés (contrast checker), et la structure sémantique contrôlée avec les outils d'audit intégrés aux navigateurs (Lighthouse). Une checklist a été suivie avant chaque livraison, couvrant notamment : présence d'un texte alternatif sur chaque image, absence de piège au clavier, contraste conforme, et labels associés à tous les champs de formulaire.

X. INSTALLATION & DÉPLOIEMENT

Cette section décrit les prérequis et les grandes étapes permettant de faire fonctionner l'application, en local pour le développement puis en production. Les commandes détaillées et le script complet sont disponibles dans le README du dépôt GitHub.

1. Prérequis

Trois éléments sont nécessaires : **Node.js v20+** (runtime du backend), **MySQL 8+** (base de données), et **Git** (récupération du code source). Chacun se vérifie simplement via node --version, mysql --version et git --version.

2. Installation locale

Après clonage du dépôt, l'installation suit un schéma classique : installation des dépendances npm côté backend et côté frontend (npm install dans chaque dossier), création d'un fichier .env à partir du modèle fourni (.env.example) contenant les identifiants de connexion à la base de données et le secret JWT, puis création de la base MySQL et exécution des migrations Sequelize pour générer les tables. Le backend démarre ensuite sur le port 3001 et le frontend sur le port 3000, ce dernier communiquant avec l'API via l'URL configurée dans ses propres variables d'environnement.
Au quotidien, le workflow de développement repose sur deux serveurs lancés en parallèle (npm run dev côté backend avec rechargement automatique, npm start côté frontend), ce qui permet de voir les modifications de code se refléter immédiatement sans redémarrage manuel.

3. Déploiement en production

Le passage en production reprend les mêmes grandes étapes avec des différences importantes : des secrets distincts de ceux du développement (jamais réutilisés), une base de données dédiée, et des variables d'environnement de production isolées. Un script de sauvegarde automatisé exporte quotidiennement la base de données (mysqldump), compresse l'archive et purge les sauvegardes de plus de 30 jours, exécuté via une tâche planifiée (cron).
Une checklist est suivie avant chaque mise en production : absence de secrets dans le dépôt Git, variables d'environnement de production correctement renseignées, npm audit sans vulnérabilité critique, migrations de base de données appliquées, et tests automatisés passés avec succès.

4. Intégration continue

Un pipeline GitHub Actions exécute automatiquement l'installation des dépendances et l'ensemble des tests à chaque push ou pull request sur les branches principales, bloquant toute fusion de code si un test échoue. Cette automatisation garantit qu'aucune régression non détectée n'atteint la branche de production.

5. Documentation pour les autres développeurs

Un README complet accompagne le dépôt, décrivant la stack technique, les prérequis, les étapes d'installation locale et les commandes de test, afin qu'un autre développeur puisse reprendre le projet de façon autonome.

XI. CONCLUSION

Bilan du projet – 15 ans condensés en quelques mois

Le projet que vous avez lu à travers ces pages n'est pas un simple exercice académique. C'est la **concrétisation d'une vision qui a germé il y a 15 ans**, quand j'ai réalisé que gérer mes recettes de cuisine était un problème universel sans solution simple. J'ai essayé C++, Java, Python, mais les technologies ne permettaient pas ce que je voulais : accéder à mes recettes de **n'importe quel endroit, n'importe quel appareil, sans installation complexe**.
Le web était la réponse manquante.
Aujourd'hui, en remettant ce dossier, je ne livre pas juste une application. Je livre une **démonstration pratique** que je maîtrise les 6 compétences du TP DWWM, et plus important, que je comprends **pourquoi les faire bien**.

Les 6 compétences – Maîtrisées, pas juste codées

Compétence 1 : Maquetter des interfaces
J'ai créé des maquettes détaillées pour chaque page, responsives sur 3 points de rupture, avec hiérarchie visuelle pensée. Les utilisateurs ne se demandent jamais "où est le bouton?" – c'est évident.

Compétence 2 : Réaliser interfaces statiques
HTML5 sémantique + CSS3 moderne + Bootstrap 5. Les interfaces ne sont pas juste belles, elles sont **accessibles** – WCAG 2.1 AA sur toute l'app. Navigation clavier complète. Contrastes 7:1. Alt text sur chaque image.

Compétence 3 : Développer la partie dynamique
React 19 hooks, gestion d'état propre, Context API pour l'authentification globale. L'app est **fluide** – aucun rechargement page, réactivité native.

Compétence 4 : Mettre en place une BDD relationnelle
3 tables normalisées 3NF, relations 1-N et N-N correctement modélisées, FK constraints avec CASCADE. La **cohérence est garantie au niveau database**, pas juste en app.

Compétence 5 : Développer accès données SQL/NoSQL
Sequelize ORM, prepared statements automatiques, aucune injection SQL possible. Les modèles incluent validations, hooks, associations. Le code est **safe by design**.

Compétence 6 : Développer composants métier serveur
Pattern RCSM strict : Route → Controller → Service → Model. Chaque couche testable seule. La logique métier est encapsulée, réutilisable, maintenable. **Pas de copier-coller**, tout est architecturé.

Évolutions futures – Penser long-terme
L'app marche aujourd'hui. Mais dans 6 mois, on pourrait vouloir :

- **Mobile app native** (React Native, même API backend)
- **Système de ratings** sur recettes
- **WebSocket temps réel** (collaboratif, voir ce que les autres favorisent)
- **Machine learning** (recommandations basées sur vos favoris)
- **Community features** (partager recettes, commentaires)

Pourquoi c'est possible? Parce que j'ai pensé **architecture**. Ajouter une mobile app ne demande pas de refonte backend – elle appelle la même API. Ajouter ML demande une microservice séparée – elle peut fonctionner indépendamment.
C'est ça la différence entre code jetable et code qui dure.

Remerciements
À **CEF** et à l'équipe pédagogique pour avoir créé un cadre de formation rigoureux où on pouvait explorer, échouer, et apprendre.
À **mes mentors/professors** pour les retours, les questions difficiles, et l'insistance que la qualité n'est pas négociable.
À ma **famille** pour la patience pendant ces mois intensifs de développement.
À la **communauté open source** – React, Express, Sequelize, Jest, et tant d'autres outils brillants que j'ai pu utiliser gratuitement.

Annexes

XII. Table des matières (Annexes)

Glossaire....42
Wireframe....48
Prototype....54
Aperçus du site....57
Arborescence....61
Arborescence frontend....61
Arborescence Backend....63

I. Glossaire

Abréviations techniques générales

Abréviation	Terme complet	Explication
API	Application Programming Interface	Interface permettant à deux applications de communiquer
CRUD	Create, Read, Update, Delete	Les 4 opérations de base sur une base de données
HTTP	HyperText Transfer Protocol	Protocole de communication web (GET, POST, PUT, DELETE)
HTTPS	HyperText Transfer Protocol Secure	HTTP sécurisé avec SSL/TLS
JSON	JavaScript Object Notation	Format de données (texte structuré)
SQL	Structured Query Language	Langage pour interroger les bases de données
REST	Representational State Transfer	Architecture API web (utilise HTTP verbes)
ORM	Object-Relational Mapping	Traducteur entre code JavaScript et base de données
JWT	JSON Web Token	Token d'authentification sécurisé (3 parties : header.payload.signature)
CORS	Cross-Origin Resource Sharing	Autorisation pour requêtes cross-domain (port différent)
RCSM	Route → Controller → Service → Model	Architecture séparation des responsabilités
DOM	Document Object Model	Représentation HTML en JavaScript
UI	User Interface	Interface utilisateur
UX	User Experience	Expérience utilisateur

Abréviations Frontend

Abréviation	Terme complet	Explication
React	React	Framework JavaScript pour interfaces
SPA	Single Page Application	Application monopage (pas de rechargement)
CSS	Cascading Style Sheets	Stylisation HTML
SCSS	Sassy CSS	Préprocesseur CSS (variables, fonctions, etc.)
HTML	HyperText Markup Language	Langage de structure web
JSX	JavaScript XML	Syntaxe React (HTML dans JavaScript)

Props	Properties	Propriétés passées aux composants React
State	État	Données changeantes dans un composant React
Hook	Crochet	Fonction React (useState, useEffect, useContext, etc.)
Axios	Axios	Client HTTP (requêtes API)
Bootstrap	Bootstrap	Framework CSS (composants prêts)
CDN	Content Delivery Network	Réseau pour distribuer des ressources
NPM	Node Package Manager	Gestionnaire de paquets JavaScript
Webpack	Webpack	Bundler (empaquetage) de modules
Babel	Babel	Transpileur (ES6 → JavaScript compatible)

Abréviations Backend

Abréviation	Terme complet	Explication
Node.js	Node.js	Runtime JavaScript côté serveur
Express	Express.js	Framework web Node.js (routage, middlewares)
Sequelize	Sequelize	ORM MySQL pour Node.js
MySQL	MySQL	Système de gestion base de données relationnelle
DB	Database	Base de données
SGBDR	Système Gestion Base Données Relationnelle	Gestionnaire de base de données
MCD	Modèle Conceptuel de Données	Représentation logique des entités et relations
MLD	Modèle Logique de Données	Diagramme visuel du schéma relationnel (intermédiaire entre MCD et MPD)
MPD	Modèle Physique de Données	Représentation concrète en tables/colonnes
FK	Foreign Key	Clé étrangère (référence à une autre table)
PK	Primary Key	Clé primaire (identifiant unique)
Middleware	Middleware	Fonction intermédiaire qui traite une requête
Service	Service	Couche métier (logique réutilisable)
Controller	Contrôleur	Orchestrateur de requête HTTP
Model	Modèle	Représentation d'une table en code

Route	Route	Endpoint HTTP (chemin de l'API)
Callback	Fonction de rappel	Fonction appelée après une action
Promise	Promise	Objet JavaScript pour asynchrone
Async/Await	Asynchrone/Attendre	Syntaxe pour gérer les Promises

Abréviations Sécurité

Abréviation	Terme complet	Explication
XSS	Cross-Site Scripting	Injection de code JavaScript malveillant
CSRF	Cross-Site Request Forgery	Requête forgée depuis un site tiers
SQL Injection	SQL Injection	Injection de code SQL malveillant
OWASP	Open Web Application Security Project	Standard de sécurité web
bcryptjs	bcryptjs	Librarie hachage de mots de passe
Hash	Hachage	Fonction unidirectionnelle (mdp → hash)
Salt	Sel	Chaîne aléatoire pour renforcer le hash
Token	Jeton	Chaîne d'authentification
Bearer	Porteur	Type de token (Bearer {token})
Authentification	Auth	Vérifier l'identité (qui es-tu ?)
Autorisation	Authz	Vérifier les permissions (as-tu le droit ?)
HTTPS	HTTPS	HTTP sécurisé
SSL/TLS	Secure Sockets Layer / Transport Layer Security	Protocole de chiffrement
Secret	Secret	Clé secrète (JWT_SECRET)

Abréviations Accessibilité et Standards

Abréviation	Terme complet	Explication
WCAG	Web Content Accessibility Guidelines	Standard d'accessibilité web
AA	Level AA	Niveau de conformité WCAG (moyen)
AAA	Level AAA	Niveau de conformité WCAG (strict)
Alt	Alternative text	Texte alternatif pour images

Aria	Accessible Rich Internet Applications	Attributs pour accessibilité
Semantic	HTML Sémantique	HTML structuré (header, nav, main, footer)
SEO	Search Engine Optimization	Optimisation moteurs de recherche
Meta tags	Balises Meta	Infos HTML pour SEO (title, description)

Abréviations du Projet Spécifique

Abréviation	Signification	Contexte
TP DWWM	Titre Professionnel Développeur Web et Web Mobile	Formation
CEF	Centre Européen de Formation	Organisme de formation
RE	Référentiel d'Évaluation	Document officiel d'évaluation
REAC	Référentiel Emploi Activités Compétences	Document officiel des compétences
MVP	Minimum Viable Product	Version minimale fonctionnelle
CCP	Certificat de Compétences Professionnelles	Certification intermédiaire
Recettes	Recettes de cuisine	Domaine du projet
Favoris	Recettes marquées	Fonctionnalité du projet
Admin	Administrateur	Rôle avec droits étendus
Membre	Utilisateur membre	Rôle standard
Upload	Téléchargement de fichier	Envoi d'image
Modale	Modal dialog	Fenêtre pop-up
Responsive	Adaptatif	S'adapte à la taille d'écran
Lazy loading	Chargement paresseux	Chargement à la demande

Abréviations Types de Données

Abréviation	Type	Utilité
INT	Integer	Nombre entier (id, temps, portions)
VARCHAR	Variable Character	Texte court (nom, email)
TEXT	Text	Texte long (description, étapes)
JSON	JSON	Données structurées (ingrédients)
ENUM	Enumeration	Valeurs prédéfinies (catégorie)

TIMESTAMP	Timestamp	Date/heure (createdAt, updatedAt)
DECIMAL	Decimal	Nombre décimal (prix)
BOOLEAN	Boolean	Vrai/Faux (isFavori)

Abréviations Fichiers et Dossiers

Abréviation	Signification	Exemple
src	Source	Dossier code source
dist	Distribution	Dossier compilé prêt
node_modules	Node Modules	Dossier dépendances
public	Public	Dossier accessible (favicon, index.html)
assets	Ressources	Images, polices, etc.
components	Composants	Composants réutilisables React
pages	Pages	Pages principales React
services	Services	Logique métier ou appels API
context	Contexte	État global React
styles	Styles	Fichiers CSS/SCSS
api	API	Configuration requêtes HTTP
models	Modèles	Modèles Sequelize
controllers	Contrôleurs	Orchestration requêtes
routes	Routes	Endpoints API
middlewares	Middlewares	Fonctions intermédiaires
.env	Environment	Variables d'environnement
.gitignore	Git Ignore	Fichiers ignorés par Git
README	ReadMe	Documentation
package.json	Package JSON	Configuration dépendances

Abréviations Environnement

Abréviation	Signification	Valeur
DB_HOST	Database Host	127.0.0.1 (localhost)
DB_PORT	Database Port	3306 (MySQL)
DB_NAME	Database Name	projet_recette
DB_USER	Database User	root
DB_PASSWORD	Database Password	(vide)

JWT_SECRET	JWT Secret	Clé secrète
PORT	Port Serveur	3001 (backend), 3000 (frontend)
NODE_ENV	Node Environment	development, production

Abréviations Verbes HTTP

Verbe	Opération	Exemple
GET	Récupérer	GET /api/recettes
POST	Créer	POST /api/recettes
PUT	Modifier	PUT /api/recettes/1
DELETE	Supprimer	DELETE /api/recettes/1
PATCH	Modifier partiellement	PATCH /api/recettes/1

Abréviations Codes HTTP

Code	Signification	Exemple
200	OK	Requête réussie
201	Created	Création réussie
204	No Content	Suppression réussie
400	Bad Request	Erreur validation
401	Unauthorized	Non authentifié
403	Forbidden	Non autorisé
404	Not Found	Ressource introuvable
500	Server Error	Erreur serveur

Résumé Rapide
Technologies principales

- **Frontend** : React 19, Bootstrap 5, SCSS, Axios
- **Backend** : Node.js, Express 5, Sequelize 6
- **Database** : MySQL 8
- **Auth** : JWT + bcryptjs
- **Versioning** : Git, GitHub

Couches de l'application

Frontend (React)
  ↓
API (REST + JWT)
  ↓
Backend (Express + RCSM)
  ├— Routes
  ├— Controllers

```
    ├── Services
    └── Models (Sequelize)
  ↓
Database (MySQL)
```

Sécurité

- JWT pour authentification
- bcryptjs pour mots de passe
- Sequelize ORM pour SQL injection
- CORS pour cross-origin
- Validation côté client ET serveur

Index alphabétique

A : API, Aria, Alt, Auth, Axios, AA, AAA
B : Bootstrap, Babel, Bearer, bcryptjs, BOOLEAN
C : CRUD, CORS, CSS, CSRF, CCP, CEF, Callback
D : DOM, DB, DBMS, DECIMAL, DELETE
E : Express, ENUM, ES6
F : FK, Frontend
G : GET, GitHub, Git
H : HTTP, HTTPS, Hash, Hook, HTML, Horizontal
J : JSON, JWT, JSX
M : MySQL, Middleware, Model, MCD, MPD
N : Node.js, NPM
O : ORM, OWASP
P : POST, Props, Promise, PK, PHP
R : React, REST, Routes, RCSM, README
S : SQL, SCSS, Service, SPA, State, SSL, Salt, SEO
T : TP, Token, TLS, Text, Timestamp
U : UI, UX, Upload, URL
V : VARCHAR, Validation
W : WCAG, Webpack
X : XSS, XML

II. Wireframe

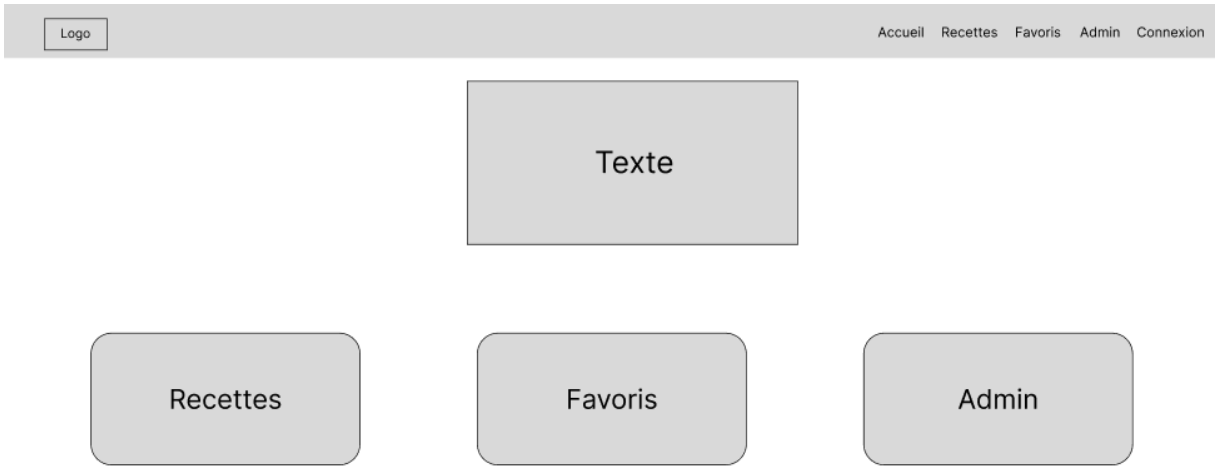


Figure 1: Page d'accueil - Wireframe

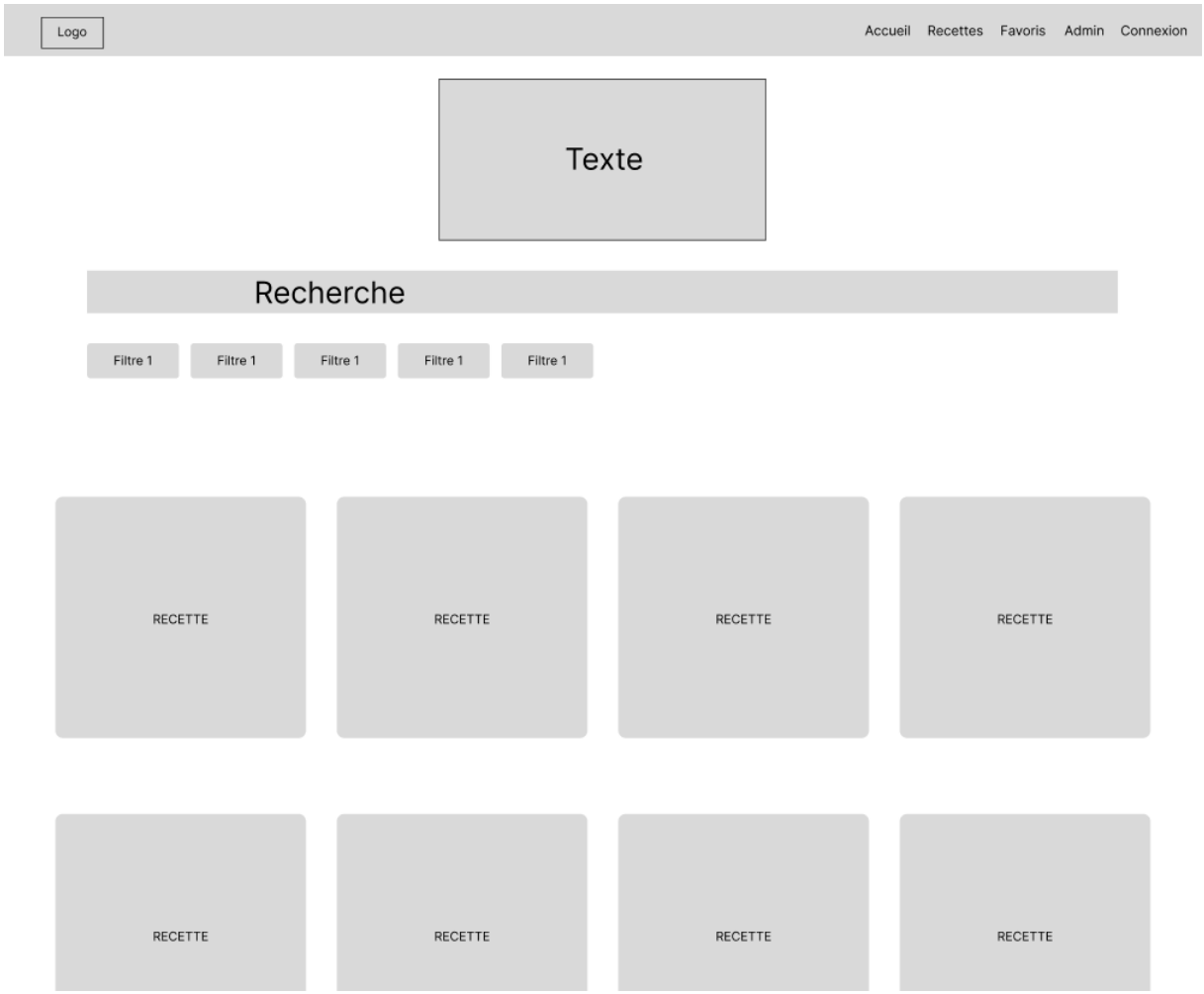


Figure 2: Page recettes - Wireframe



Figure 3: Page favoris - Wireframe



Figure 4: Page admin - Wireframe



Figure 5: Page connexion - Wireframe



Figure 6: Modale Recette

III. Prototype



Figure 7 : Page d'accueil - Prototype

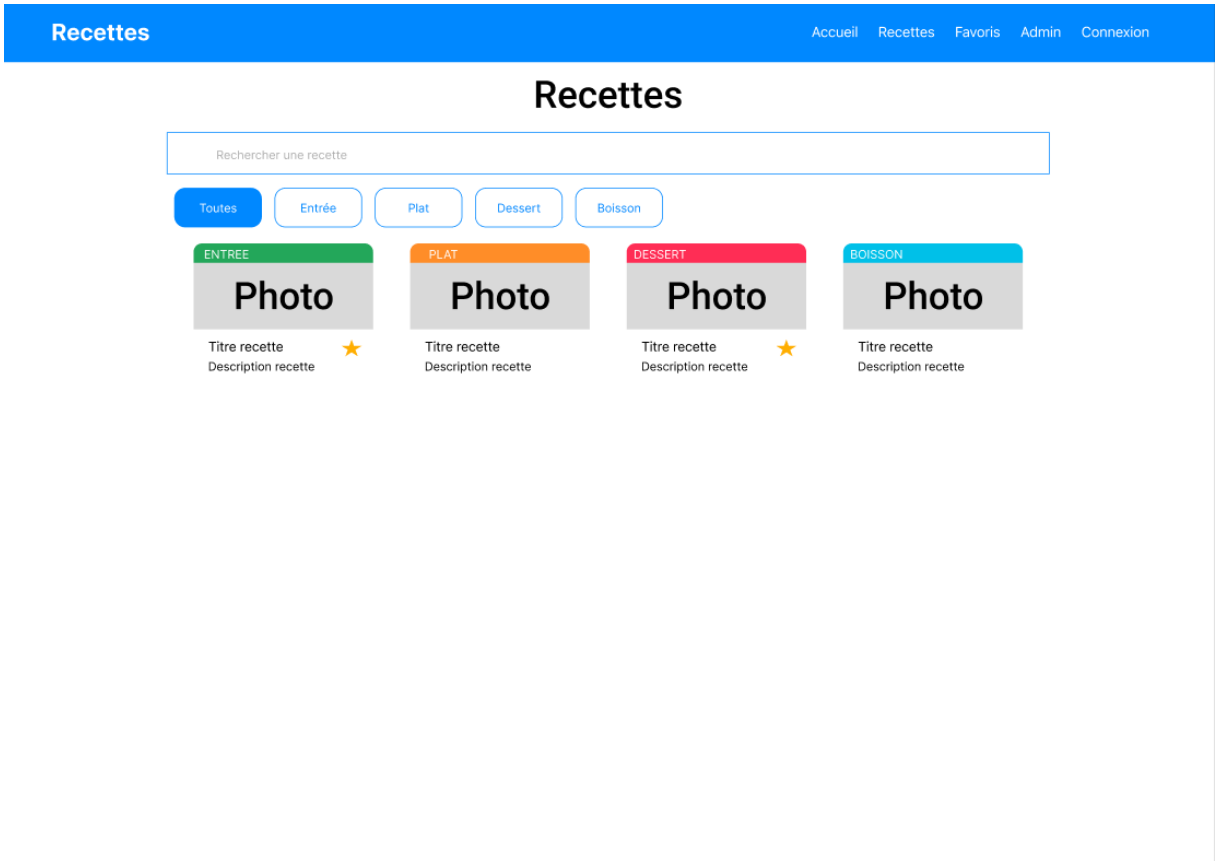


Figure 8: Page recettes - Prototype

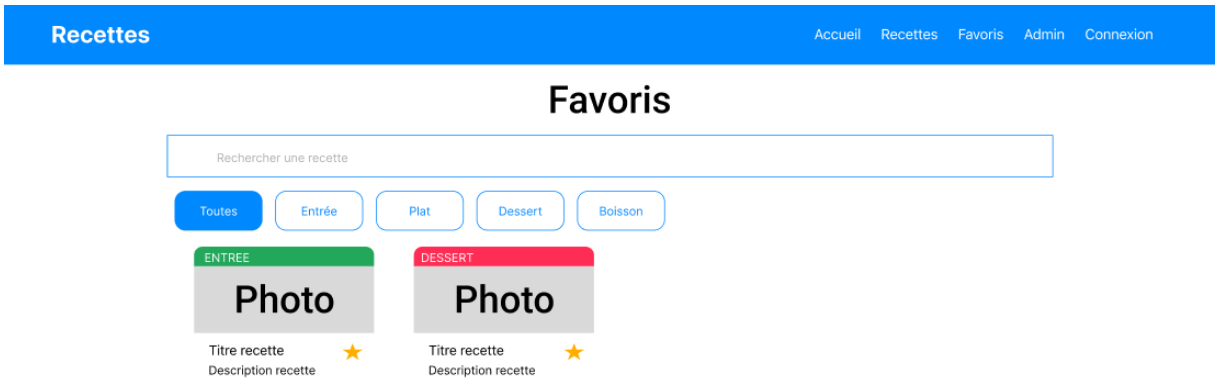


Figure 9 : Page favoris - Prototype



Figure 10 : Page Admin - Prototype

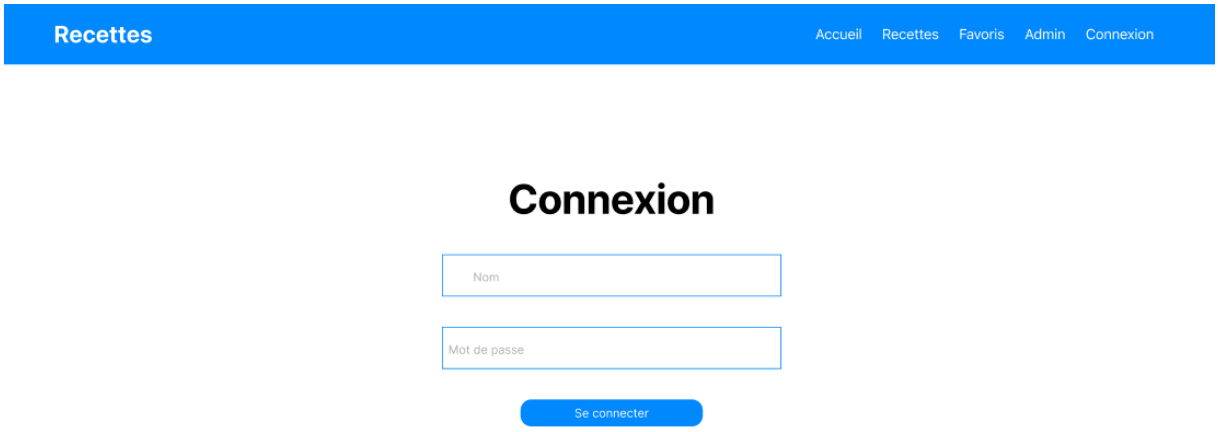


Figure 11 : Page Connexion - Prototype

IV. Aperçus du site

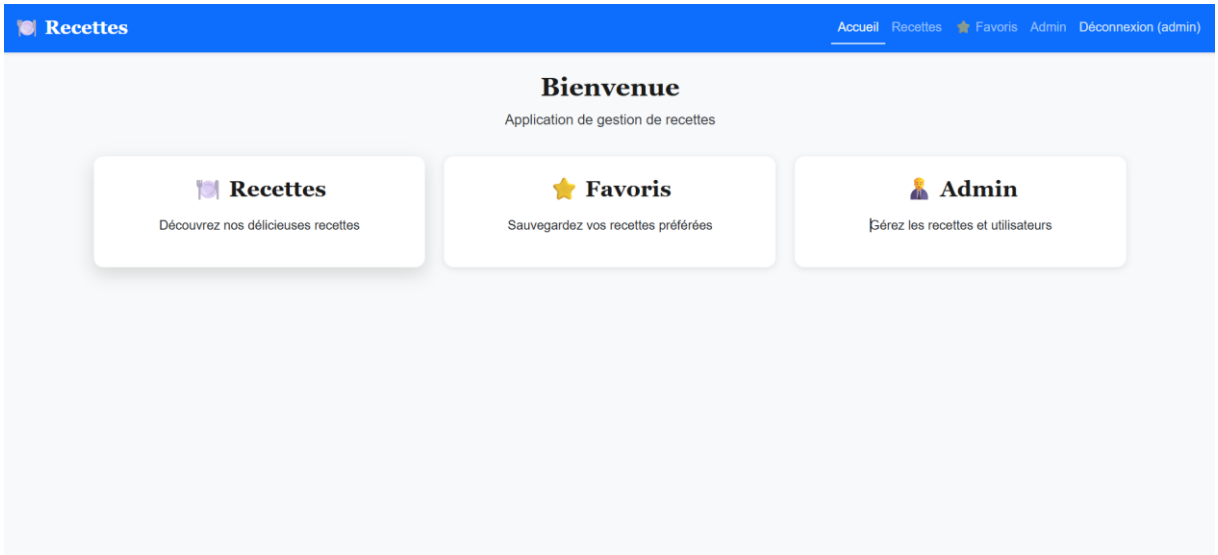


Figure 12 : Page d'accueil – Aperçu

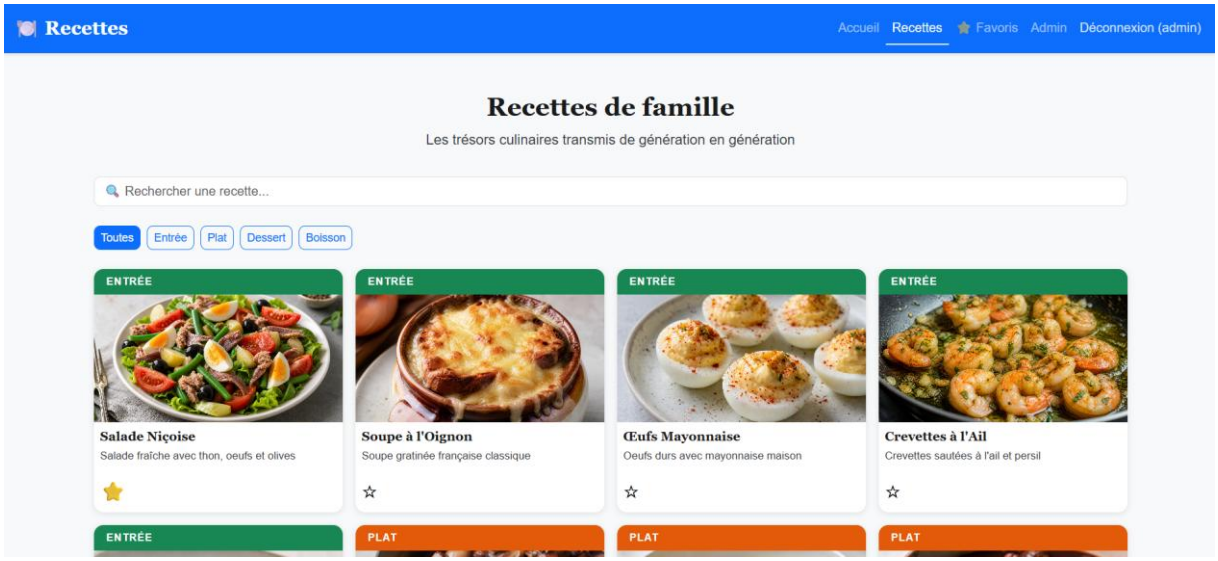


Figure 13 : Page recettes

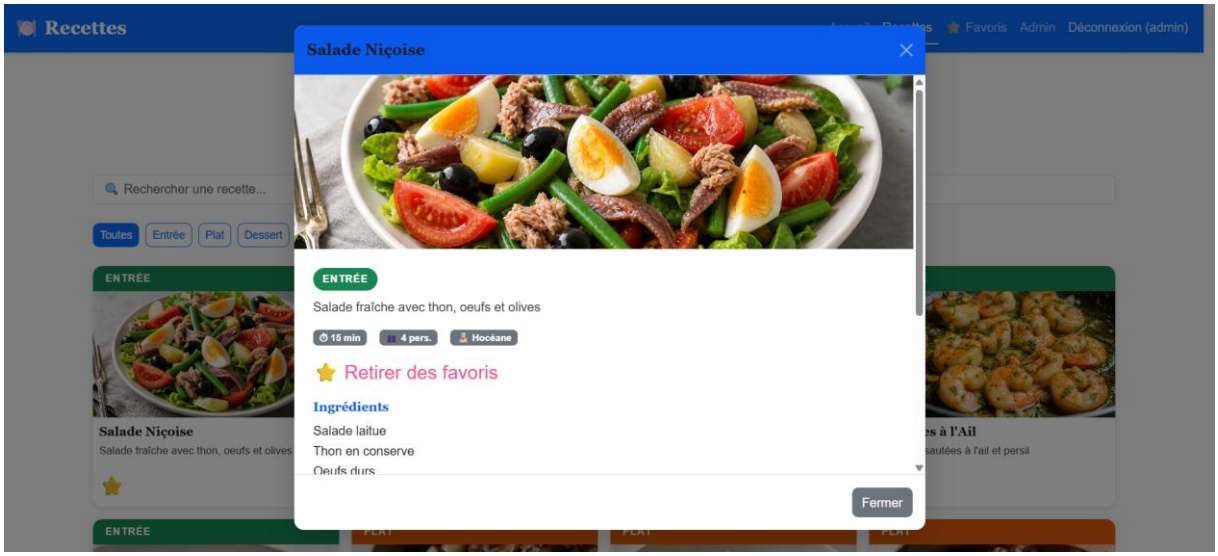


Figure 14 : Modale Recette

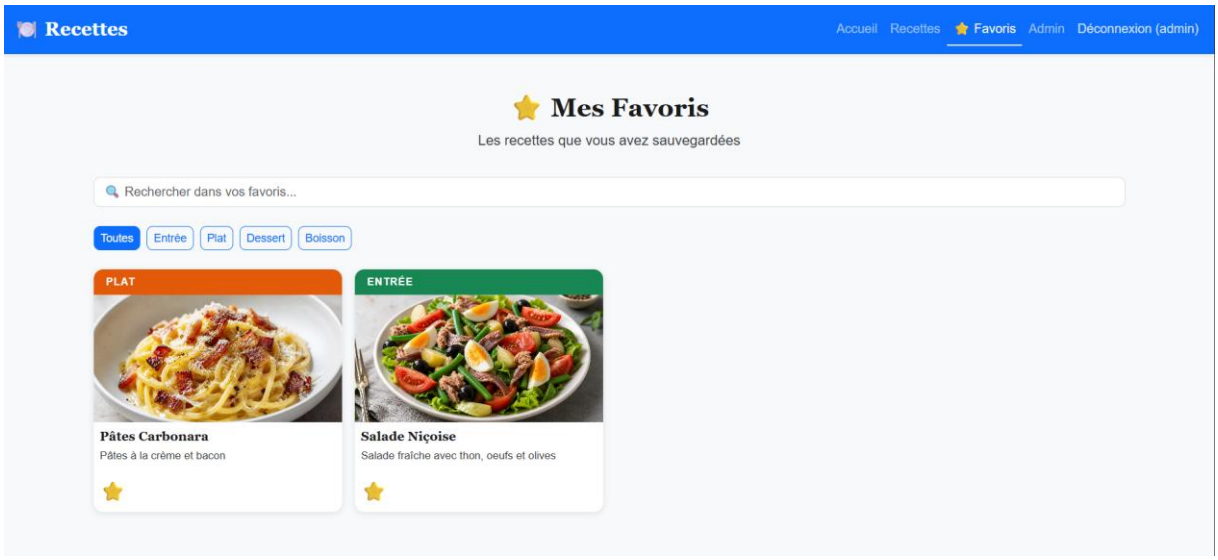


Figure 15 : Page favoris

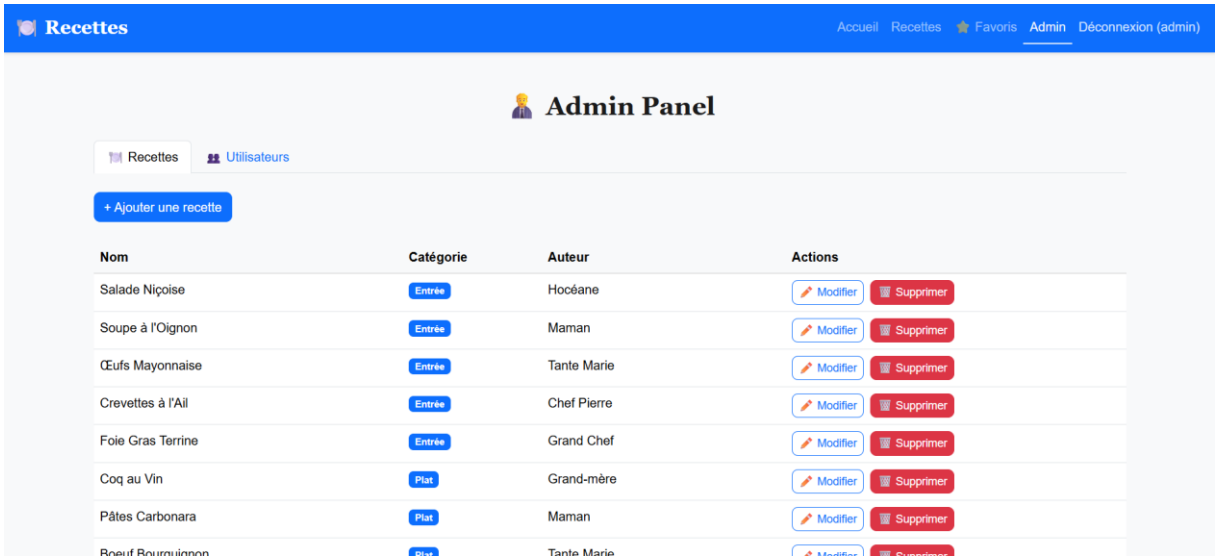


Figure 16 : Page Admin

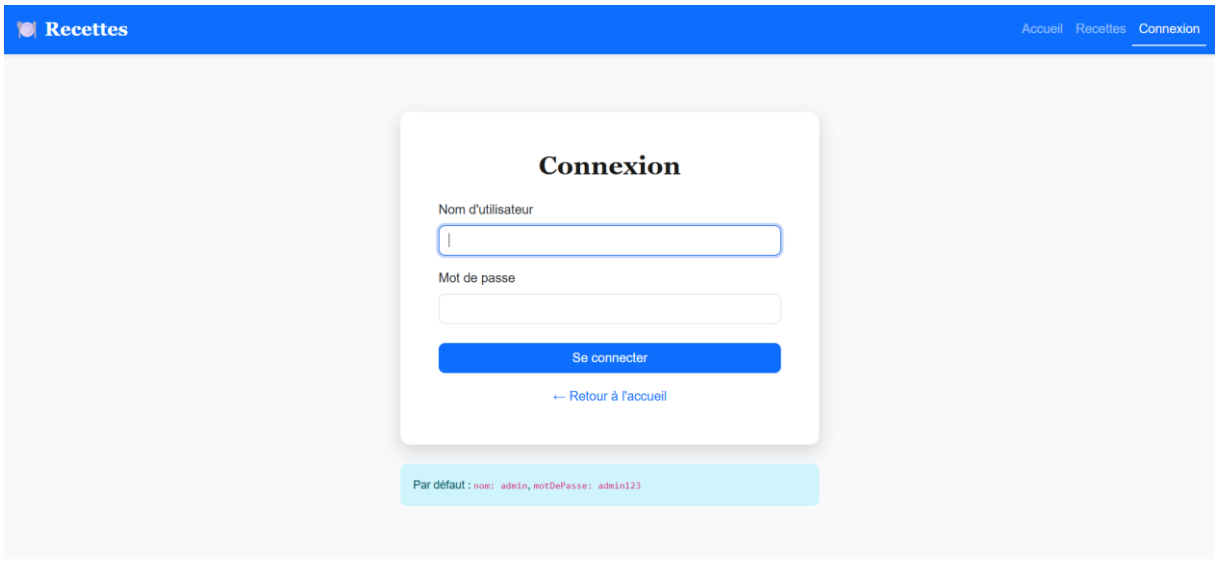


Figure 17 : Page connexion

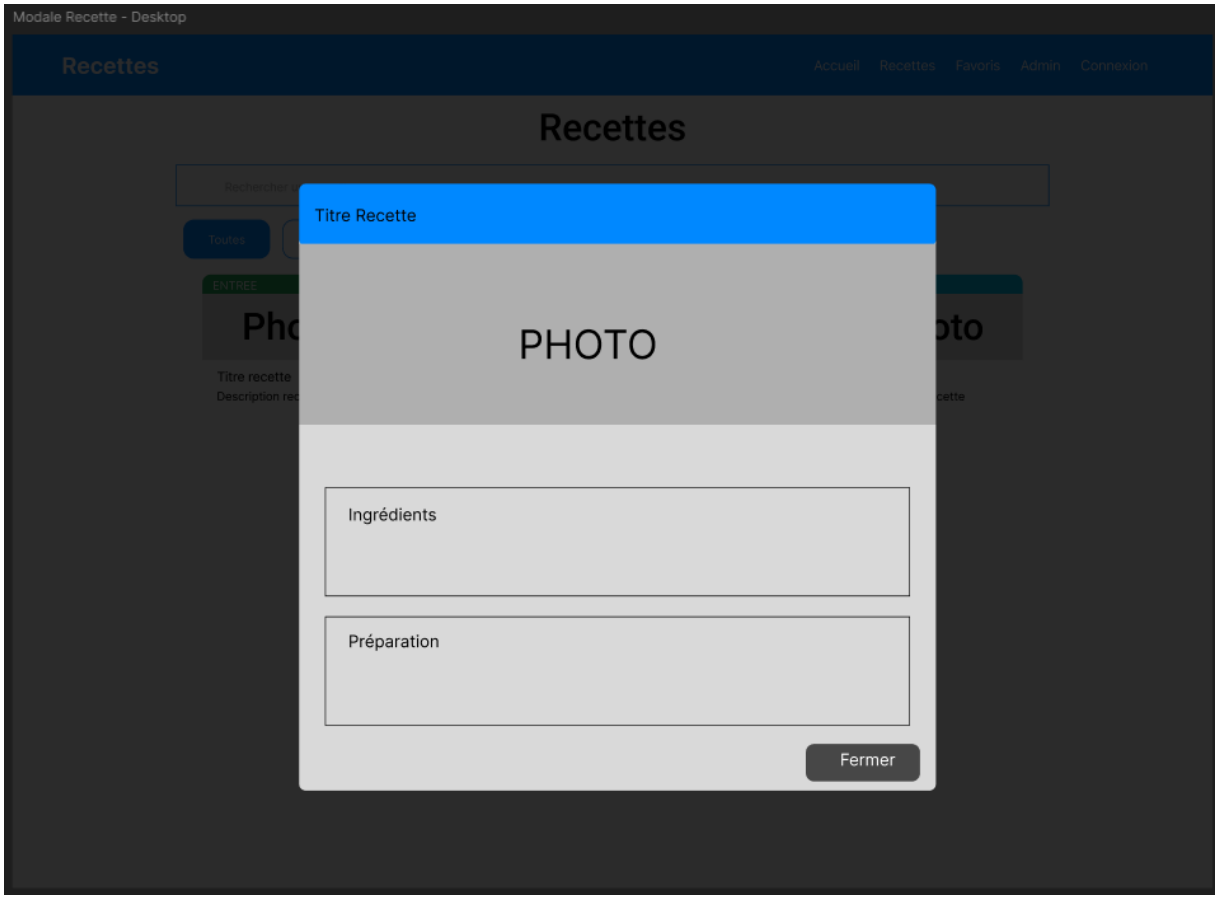


Figure 18 : Modale Recette

V. Arborescence

6. Arborescence frontend

```
frontend/
│
├── public/                    # Fichiers statiques publics
│   ├── favicon.ico            # Icône du site
│   ├── index.html             # Fichier HTML principal
│   └── manifest.json          # Configuration PWA
│
├── src/                       # Code source
│   │
│   ├── pages/                 # Pages principales (React Router)
│   │   ├── Accueil.js         # Page d'accueil
│   │   ├── Recettes.js        # Listing recettes (recherche + filtres)
│   │   ├── Favoris.js         # Page mes favoris
│   │   ├── Login.js           # Page connexion
│   │   ├── Admin.js           # Panel administration
│   │   ├── NotFound.js        # Page 404
│   │   └── ProtectedRoute.js  # Wrapper route protégée
│   │
│   ├── components/            # Composants réutilisables
│   │   ├── RecetteCard.js     # Carte recette (grid)
│   │   ├── RecetteModal.js    # Modale détail recette
│   │   ├── NavBar.js          # Barre navigation
│   │   ├── Footer.js          # Pied de page
│   │   ├── LoadingSpinner.js  # Spinner chargement
│   │   └── AlertMessage.js    # Messages d'alerte
│   │
│   ├── context/               # État global (Context API)
│   │   ├── AuthContext.js     # Contexte authentification
│   │   └── AuthProvider.js    # Provider authentification
│   │
│   ├── api/                   # Configuration API
│   │   └── index.js           # Configuration Axios + intercepteur JWT
│   │
│   ├── services/              # Appels API métier
│   │   ├── recettesService.js # Appels API recettes
│   │   ├── favorisService.js  # Appels API favoris
│   │   └── authService.js     # Appels API authentification
│   │
│   ├── styles/                # Feuilles de style SCSS
│   │   ├── _variables.scss    # Variables couleurs, fonts, breakpoints
│   │   ├── _global.scss       # Styles globaux (reset, body, etc.)
│   │   ├── _cards.scss        # Styles cartes recettes
│   │   ├── _form.scss         # Styles formulaires
│   │   ├── _navbar.scss       # Styles navigation
│   │   ├── _modal.scss        # Styles modales
│   │   └── App.scss           # Styles App principal
│   │
│   ├── App.js                 # Composant principal (routage)
│   ├── App.css                # CSS App
│   ├── index.js               # Entrée React
│   └── index.css              # CSS global
```

```
│
├── .gitignore                # Fichiers ignorés Git
├── .env                      # Variables environnement (NE PAS COMMIT)
├── .env.example              # Exemple .env
├── package.json              # Dépendances et scripts
├── package-lock.json         # Lock dépendances
├── README.md                 # Documentation
└── public/index.html         # HTML principal
```

7. Arborescence Backend

```
backend/
│
├── config/                    # Configuration
│   └── database.js              # Configuration Sequelize
│
├── models/                    # Modèles Sequelize
│   ├── Utilisateur.js           # Modèle utilisateur
│   ├── Recette.js               # Modèle recette
│   ├── Favori.js                # Modèle favori
│   └── index.js                 # Initialisation + associations
│
├── controllers/               # Contrôleurs (orchestration)
│   ├── authController.js          # Contrôleur authentification
│   │   ├── login()
│   │   ├── getMe()
│   │   ├── getUtilisateurs()
│   │   ├── updateUtilisateur()
│   │   └── deleteUtilisateur()
│   │
│   ├── recettesController.js      # Contrôleur recettes
│   │   ├── getAllRecettes()
│   │   ├── getRecetteById()
│   │   ├── createRecette()
│   │   ├── updateRecette()
│   │   ├── deleteRecette()
│   │   └── deletePhoto()
│   │
│   └── favorisController.js       # Contrôleur favoris
│       ├── getFavoris()
│       ├── addFavori()
│       ├── removeFavori()
│       └── isFavori()
│
├── services/                  # Services (logique métier)
│   ├── recettesService.js         # Logique recettes
│   │   ├── getAllRecettes()
│   │   ├── getRecetteById()
│   │   ├── createRecette()
│   │   ├── updateRecette()
│   │   └── deleteRecette()
│   │
│   └── favorisService.js          # Logique favoris
│       ├── getFavoris()
│       ├── addFavori()
│       ├── removeFavori()
│       └── isFavori()
│
├── routes/                    # Routes API
│   ├── auth.js                  # Routes /api/auth
│   │   ├── POST /api/auth/login
│   │   ├── GET /api/auth/me
│   │   ├── GET /api/auth/utilisateurs
│   │   ├── PUT /api/auth/utilisateurs/:id
```

```
│   │   └── DELETE /api/auth/utilisateurs/:id
│   │
│   ├── recettes.js              # Routes /api/recettes
│   │   ├── GET /api/recettes
│   │   ├── GET /api/recettes/:id
│   │   ├── POST /api/recettes
│   │   ├── PUT /api/recettes/:id
│   │   ├── DELETE /api/recettes/:id
│   │   └── DELETE /api/recettes/:id/photo
│   │
│   └── favoris.js               # Routes /api/favoris
│       ├── GET /api/favoris
│       ├── GET /api/favoris/:id
│       ├── POST /api/favoris/:id
│       └── DELETE /api/favoris/:id
│
├── middlewares/                 # Middlewares
│   ├── auth.js                  # JWT validation
│   │   ├── isAuthenticated()    # Vérifie JWT
│   │   └── isAdmin()            # Vérifie admin
│   │
│   └── upload.js                # Multer configuration
│       └── uploadPhoto()        # Validation upload image
│
├── uploads/                     # Images uploadées
│   ├── 1701234567890-image.jpg
│   ├── 1701234568891-image.png
│   └── ... (autres images)
│
├── .env                         # Variables environnement
│   ├── DB_HOST=127.0.0.1
│   ├── DB_PORT=3306
│   ├── DB_NAME=projet_recette
│   ├── DB_USER=root
│   ├── DB_PASSWORD=
│   ├── JWT_SECRET=votre_secret_complexe
│   ├── PORT=3001
│   └── NODE_ENV=development
│
├── seed.sql                     # Script création BDD + données test
│   ├── CREATE DATABASE projet_recette
│   ├── CREATE TABLE utilisateurs (...)
│   ├── CREATE TABLE recettes (...)
│   ├── CREATE TABLE favoris (...)
│   ├── INSERT INTO utilisateurs VALUES (1, 'admin', [hash], 'admin')
│   └── INSERT INTO recettes VALUES (20 recettes de test)
│
├── index.js                     # Serveur Express principal
├── package.json                 # Dépendances
├── package-lock.json            # Lock dépendances
├── .gitignore                   # Fichiers ignorés
├── .env.example                 # Exemple .env
└── README.md                    # Documentation
```